

EGZAMIN WSTĘPNY

studia magisterskie

SZTUCZNA INTELIGENCJA

Notatki do powtórki

*wybór najważniejszego materiału,
wy tłumaczony spokojnie i po ludzku*

Spis treści

1	Co powtarzać i dlaczego	1
1.1	Punktacja i co z niej wynika	1
1.2	Ranking: od czego zacząć	1
1.3	Czego świadomie tu nie ma	2
2	Prawdopodobieństwo i statystyka	3
2.1	Co wolno bez żadnych założeń, a co wymaga niezależności	3
2.2	Warunkowanie, prawdopodobieństwo całkowite i wzór Bayesa	4
2.3	Rozkłady, które naprawdę wracają	5
2.4	Rozkład wykładniczy i jego trzy własności	6
2.5	Nierówności Markowa i Czebyszewa	7
2.6	Centralne twierdzenie graniczne	8
2.7	Metoda największej wiarygodności	8
2.8	Wartość p, poziom istotności i błędy dwóch rodzajów	9
2.9	Nieobciążoność, zgodność i efektywność	10
3	Algebra liniowa	11
3.1	Liczby zespolone w postaci wykładniczej	11
3.2	Jak rozstrzygnąć, czy zbiór jest podprzestrzenią	12
3.3	Jądro i obraz przekształcenia liniowego	13
3.4	Wartości i wektory własne	14
3.5	Wyznacznik i jego własności	15
3.6	Macierze symetryczne i określoność formy	16
4	Matematyka dyskretna	18
4.1	Rzędy wzrostu i notacje asymptotyczne	18
4.2	Rekurencje liniowe	20
4.3	Twierdzenie o rekurencji uniwersalnej	20
4.4	Liczby Fibonacciego i złoty podział	21
4.5	Funkcje tworzące	21
4.6	Indukcja matematyczna	23
4.7	Współczynniki dwumianowe i trójkąt Pascala	23
4.8	Podstawowe schematy zliczania	24
4.9	Zasada szufladkowa Dirichleta	25

4.10	Arytmetyka modularna	25
4.11	Chińskie twierdzenie o resztach	26
4.12	Działanie grupy na zbiorze	27
4.13	Grafy: fakty, które wracają	28
4.14	Kolorowanie grafów	29
5	Analiza matematyczna	30
5.1	Zbieżność ciągów bez rachunku	30
5.2	Szeregi z parametrem i końce przedziału zbieżności	31
5.3	Kryteria zbieżności szeregów w praktyce	32
5.4	Twierdzenie Lagrange’a jako maszyna do nierówności	33
5.5	Funkcje wypukłe	33
5.6	Wzór Taylora i rozwinięcia w zerze	34
5.7	Kanoniczne kontrprzykłady	34
5.8	Funkcje wielu zmiennych	35
6	Algorytmy i struktury danych	36
6.1	Jak czytać nieznany pseudokod	36
6.2	Duże O jako obietnica, która łatwo wybacza	37
6.3	Podstawowe struktury danych	38
6.4	Drzewa poszukiwań binarnych i ich uogólnienia	38
6.5	Drzewa AVL i ich wysokość	39
6.6	Tablice haszujące i słowniki	40
6.7	Sortowania: tylko to, co rozstrzyga stwierdzenia	41
6.8	Selekcja, czyli k-ty element bez sortowania	42
6.9	Techniki algorytmiczne	42
6.10	Programowanie dynamiczne i najdłuższy wspólny podciąg	43
6.11	Programowanie dynamiczne w pozostałych zadaniach	44
6.12	Przeglądy grafu, porządki i spójność	45
6.13	Drzewa rozpinające, najkrótsze ścieżki i wagi ujemne	46
6.14	Wszystkie pary, przepływy i skojarzenia	47
6.15	Wyszukiwanie wzorca w tekstach	48
6.16	Poprawność i koszt zamortyzowany	49
7	Python	50
7.1	Zmienna jest etykietą, nie pudełkiem	50
7.2	Kopia, widok i maski w numpy	51
7.3	Wyrażenia listowe i koszt kontenerów	52
7.4	Wyjątki: TypeError kontra ValueError	53
7.5	Trzy drobiazgi, które zaskakują	53

8	Metody numeryczne	55
8.1	Metody iteracyjne rozwiązywania równań nieliniowych	55
8.2	Uwarunkowanie zadania i stabilność algorytmu	57
8.3	Utrata cyfr znaczących	57
8.4	Całkowanie numeryczne	57
8.5	Interpolacja wielomianowa	58
8.6	Rozkład LU i po co się go robi	58

ROZDZIAŁ 1

Co powtarzać i dlaczego

Te notatki są wyborem, nie streszczeniem. Oficjalny zakres obejmuje 55 zagadnień z siedmiu przedmiotów; przerobienie wszystkiego na równym poziomie jest pracą na miesiące i nie jest optymalne punktowo. Wybór opiera się na porównaniu zakresu z arkuszem z 17 września 2025 — bo zakres mówi, co *może* się pojawić, a arkusz, co pojawiło się *naprawdę* i w jakiej formie.

Te dwie rzeczy się rozjeżdżają. Zakres wymienia długości krzywych i pola powierzchni obrotowych, a na arkuszu nie było ani jednego takiego zadania. Za to dwa zadania wymagały sprawnego porównywania rzędów wzrostu ciągów, czego zakres nie wyróżnia wcale. O wartości tematu decyduje więc nie obecność w spisie, ale to, ile stwierdzeń prawda/fałsz da się z niego ułożyć.

1.1 Punktacja i co z niej wynika

Za zadanie: 4 trafienia dają 4 pkt, 3 trafienia 2,5 pkt, 2 trafienia 1,5 pkt, 1 trafienie 0,5 pkt. Jeśli k podpunktów znasz pewnie, a resztę zgadujesz:

Pewnych podpunktów	0	1	2	3	4
Oczekiwane punkty	1,56	2,06	2,63	3,25	4
Zysk z kolejnego	—	+0,50	+0,56	+0,63	+0,75

Zysk z kolejnego rozstrzygniętego podpunktu *rośnie*, więc opłaca się domykać tematy, a nie liznąć wszystkiego po trochu: 21 zadań na 4/4 daje około 90 punktów, a „w każdym zadaniu wiem trzy z czterech” tylko 81. Ślepe zgadywanie całego arkusza daje 39 punktów — to Twoje zero. Punktów ujemnych nie ma, a pusta kratka liczy się jak błąd, więc zawsze wypełniasz wszystkie 100 kratek.

1.2 Ranking: od czego zacząć

Kolejność wynika z trzech kryteriów: **zasięgu** (ile zadań obsługuje dana umiejętność), **gęstości pułapek** (ile fałszywych zdań da się z tematu zrobić) i **kosztu** nauki w stosunku do punktów.

- Rzędy wzrostu i asymptotyka** — obsługuje zadania w trzech działach naraz (ciągi, szeregi, złożoność) i rozstrzyga się je w pamięci.
- Warunki zbieżności szeregów, zwłaszcza brzegi przedziału** — najwyższa gęstość pułapek w całym materiale.
- Wartości i wektory własne, krotności, diagonalizowalność** — rdzeń algebry i baza twierdzenia spektralnego.
- Czytanie nieznanego pseudokodu** — umiejętność, nie temat; na arkuszu było zadanie z algorytmem, którego nikt wcześniej nie widział.

5. **Co wymaga niezależności, a co nie** — najproduktywniejsze źródło fałszywych zdań w prawdopodobieństwie.
6. **Programowanie dynamiczne, zwłaszcza oba przypadki równania LCS** — pułapka jest zawsze ta sama: wzór z drugiej gałęzi.
7. **Twierdzenie Lagrange’a jako maszyna do nierówności** — tanie i płodne.
8. **Orbita i stabilizator** — wąskie, ale wysoce przewidywalne.
9. **Metody iteracyjne: rzędy zbieżności i założenia** — całe metody numeryczne sprowadzają się w praktyce do kilku takich rozróżnień.

1.3 Czego świadomie tu nie ma

Luki są zamierzone. Pominęte albo skrócone do akapitu: zastosowania geometryczne całki (długości krzywych, powierzchnie obrotowe, środki ciężkości), mechanika całkowania (ułamki proste, podstawienia Eulera), statystyka opisowa, regresja liniowa, konstrukcja bazy Jordana, ortogonalizacja Grama-Schmidta, metaprogramowanie w Pythonie oraz programowanie obiektowe. Wszystko to jest opracowane w pełnej wersji repetytorium.

- ▷ *Sposób pracy, który polecam: przeczytaj rozdział, potem zasłoń tekst i powiedz własnymi słowami, czym jest każde wyróżnione pojęcie i od jakiego założenia zależy. Jeżeli potrafisz to zrobić na głos, rozstrzygniesz też stwierdzenie na egzaminie.*

ROZDZIAŁ 2

Prawdopodobieństwo i statystyka

prawdopodobieństwo i statystyka: Ten dział jest dla autorów testu prawda/fałsz wyjątkowo wdzięczny, bo prawie każdy wzór ma tu założenie, które można cicho usunąć. Zdanie „wariancja sumy jest sumą wariancji” wygląda niewinnie, a jest fałszywe, dopóki nie dopisze się słowa „niezależnych”. Druga fabryka fałszywych stwierdzeń to interpretacje: czym właściwie jest wartość p , co mówi centralne twierdzenie graniczne, czy nieskorelowanie to już niezależność. Trzecia to rozkłady — wystarczy podstawić wariancję rozkładu geometrycznego tam, gdzie mowa o wykładniczym, i zdanie przestaje być prawdziwe, choć wygląda identycznie. Dlatego w tym rozdziale mało liczymy, a dużo pilnujemy założeń i znaczeń.

2.1 Co wolno bez żadnych założeń, a co wymaga niezależności

Jeśli miałbyś powtórzyć z całego prawdopodobieństwa jedną rzecz, to właśnie tę. Podmiana jest tu niewidoczna: wzór zostaje ten sam, znika tylko jedno słowo obok.

liniowość wartości oczekiwanej

Wartość oczekiwaną najlepiej wyobrażać sobie jako środek masy rozkładu. Gdyby rozłożyć masę wzdłuż osi liczbowej proporcjonalnie do prawdopodobieństw, wartość oczekiwana jest punktem, w którym taka sztaba się zbalansuje. Stąd bierze się jej najważniejsza własność:

$$E(aX + bY) = aEX + bEY.$$

Podkreślam słowo, które jest tu całą treścią: *zawsze*. Nie ma żadnego założenia o niezależności, o nieskorelowaniu ani o wspólnym rozkładzie — wystarczy, że obie wartości oczekiwane istnieją. Zmienne mogą być powiązane w najgorszy możliwy sposób, na przykład $Y = -X$ albo $Y = X^{17}$, a wzór nadal obowiązuje. Powód jest prosty: wartość oczekiwana to suma wartości przemnożonych przez prawdopodobieństwa, więc rozbicie $E(aX + bY)$ jest tylko przegrupowaniem składników. Praktycznie znaczy to, że gdy pytanie dotyczy wyłącznie wartości oczekiwanej sumy, o zależnościach nie musisz wiedzieć nic — rozbijasz sumę i liczysz składnik po składniku.

iloczyn i wariancja, czyli tam gdzie założenie jest konieczne

Z iloczynem jest zupełnie inaczej. Równość $E(XY) = EXEY$ nie jest własnością wartości oczekiwanej — jest *definicją* tego, że zmienne są nieskorelowane. Widać to po przepisaniu wzoru na kowariancję, $\text{Cov}(X, Y) = E(XY) - EXEY$, która jest dokładnie miarą tego, o ile obie strony się różnią. Jeżeli w zdaniu nie napisano ani o niezależności, ani o nieskorelowaniu, takie zdanie jest fałszywe.

Podobnie jest z wariancją, gdzie wzór obowiązujący bez założeń ma trzy składniki:

$$\text{Var}(X + Y) = \text{Var } X + \text{Var } Y + 2 \text{Cov}(X, Y).$$

Wariancja nie jest więc liniowa i to jest sedno sprawy. Dopiero gdy kowariancja znika, zostaje samo dodawanie — do czego wystarcza nieskorelowanie, a w szczególności niezależność. Współczynniki wychodzą przed nawias w kwadratach, $\text{Var}(aX) = a^2 \text{Var } X$, bo wariancja mierzy kwadrat odchylenia.

! Znak minus przy zmiennej nie przenosi się na wariancję. Dla niezależnych X i Y zachodzi $\text{Var}(X - Y) = \text{Var } X + \text{Var } Y$, a nie różnica, bo $\text{Var}(-Y) = (-1)^2 \text{Var } Y = \text{Var } Y$. Wariancja mierzy rozrzut, a rozrzutu nie da się zmniejszyć przez odejmowanie czegoś losowego: odjęcie niezależnej zmiennej dokładnie niepewności tyle samo, ile dołożyłoby jej dodanie. Zdanie o „różnicy wariancji” brzmi symetrycznie i dlatego wiarygodnie, a jest fałszywe.

niezależność a nieskorelowanie

Zależność jest jednokierunkowa: niezależność zawsze daje nieskorelowanie, ale nie odwrotnie. Korelacja mierzy wyłącznie *liniowy* związek, więc zmienne mogą być powiązane bardzo ściśle, byle nie po linii prostej, a ona tego nie zobaczy.

Najprostszy kontrprzykład: niech X ma rozkład symetryczny względem zera, na przykład jednostajny na $[-1, 1]$, i niech $Y = X^2$. Znając X znasz Y dokładnie, bez śladu losowości. A jednak kowariancja wynosi $E(X^3) - EX E(X^2) = 0$, bo oba składniki znikają z symetrii: dla każdej wartości dodatniej istnieje równie prawdopodobna przeciwna, więc wszystko skraca się parami. Obrazowo — wzrost X w prawo podnosi Y , ale wzrost X w lewo podnosi Y tak samo, więc „średni kierunek” zależności jest zerowy. Korelacja mierzy tylko ten kierunek i dlatego ogłasza brak związku tam, gdzie związek jest doskonały.

trzy poziomy założeń: Bez żadnych założeń wolno: rozbić wartość oczekiwaną sumy oraz użyć pełnego wzoru na wariancję sumy z kowariancją. Przy nieskorelowaniu dodatkowo wolno: rozbić wartość oczekiwaną iloczynu i dodać wariancje. Dopiero pełna niezależność pozwala mnożyć prawdopodobieństwa, czyli pisać $P(X \in A, Y \in B) = P(X \in A)P(Y \in B)$. Autorzy testu najczęściej podnoszą stwierdzenie o jeden poziom wyżej, niż na to pozwalają podane założenia.

2.2 Warunkowanie, prawdopodobieństwo całkowite i wzór Bayesa

warunkowanie jako zawężenie przestrzeni

Za definicją $P(A \mid B) = P(A \cap B)/P(B)$ stoi prosta intuicja: warunkowanie to zawężenie przestrzeni. Dowiedziawszy się, że zaszło B , wyrzucasz scenariusze z nim sprzeczne i zostajesz w mniejszym świecie, w którym prawdopodobieństwa muszą się nadal sumować do jedynki — i tym właśnie jest dzielenie przez $P(B)$.

Gdy zdarzenia $A_1, \dots, A_n$ rozbijają przestrzeń na rozłączne przypadki, prawdziwy jest wzór na prawdopodobieństwo całkowite,

$$P(B) = \sum_{i=1}^n P(B \mid A_i) P(A_i),$$

który czyta się jak przepis: podziel świat na przypadki, w każdym policz swoje prawdopodobieństwo osobno, a potem sklej wyniki, ważąc je prawdopodobieństwem samego przypadku. To zwykła średnia ważona — i tym wzorem jest prawie każde zadanie z losowaniem urny, a potem kuli z tej urny.

wzór Bayesa

Wzór Bayesa odwraca kierunek wnioskowania:

$$P(A_i | B) = \frac{P(B | A_i) P(A_i)}{P(B)}.$$

Świat zwykle łatwo opisać od przyczyny do skutku: wiemy, jak często chory człowiek dostaje dodatni wynik testu. Interesuje nas jednak kierunek odwrotny — widzimy dodatni wynik i pytamy, czy pacjent jest chory. Ceną za to odwrócenie jest konieczność podania, jak częsta jest przyczyna, zanim cokolwiek zaobserwowaliśmy.

dlaczego test medyczny zaskakuje

Choroba dotyczy jednej osoby na tysiąc, test wykrywa ją u chorych w 99% przypadków i myli się u zdrowych w 1%. Wygląda bardzo dobrze — jakie więc jest prawdopodobieństwo, że osoba z dodatnim wynikiem jest chora?

Najłatwiej policzyć to na populacji stu tysięcy osób, bez ułamków. Chorych jest sto, z czego test wykryje dziewięćdziesiąt dziewięć. Zdrowych jest blisko sto tysięcy, a jeden procent z nich, czyli około tysiąca osób, dostanie fałszywie dodatni wynik. Dodatnich wyników jest zatem około $99 + 999 = 1098$, a prawdziwie chorych wśród nich tylko 99 — niecałe 9%.

Puenta: przy rzadkiej chorobie *większość wyników dodatnich jest fałszywa*, i to nie z powodu słabości testu. Grupa zdrowych jest tysiąc razy liczniejsza, więc drobny odsetek jej pomyłek przewyższa liczbę trafień w grupie maleńkiej.

! Nie wolno mieszać $P(A | B)$ z $P(B | A)$; większość paradoksów działu polega na ich cichej podmianie. Wyżej $P(\text{dodatni} | \text{chory}) = 0,99$, a $P(\text{chory} | \text{dodatni}) \approx 0,09$.

2.3 Rozkłady, które naprawdę wracają

Najcenniejsza umiejętność to nie pamiętanie gęstości, lecz rozpoznanie rozkładu z opisu słownego — bo gdy wiesz, o który chodzi, wartość oczekiwaną i wariancję po prostu odczytujesz.

cztery rozkłady dyskretne

Wszystko zaczyna się od rozkładu **dwupunktowego**: jedno doświadczenie o dwóch wynikach, sukces z prawdopodobieństwem p albo porażka. Sam rzadko jest tematem zadania, ale jest cegiełką dwóch następnych.

Rozkład **dwumianowy** opisuje sytuację, w której powtarzasz to samo doświadczenie ustaloną z góry liczbę n razy, niezależnie i przy niezmiennym p , a interesuje cię liczba sukcesów — jak w „rzucamy monetą dziesięć razy”. Rozkład **geometryczny** dotyczy tej samej próby, ale liczby powtórzeń nie ustalasz: powtarzasz, aż zdarzy się sukces, i pytasz, jak długo trzeba było czekać. Zdradza go słowo „pierwszy”. Jest więc dokładnie odwrotnie niż w dwumianowym: liczba prób jest losowa, a sukcesów znana i równa jedynce.

Rozkład **Poissona** modeluje liczbę zdarzeń w ustalonym oknie czasu lub przestrzeni, gdy są rzadkie, niezależne, a średnia intensywność stała: awarie serwera na dobę, zgłoszenia do infolinii na godzinę. Jest granicą dwumianowego przy rosnącym n i malejącym p o stałym iloczynie np — stąd pasuje do wielu okazji, z których każda jest bardzo mało prawdopodobna.

trzy rozkłady ciągłe

W rozkładzie **jednostajnym** nic w przedziale nie jest wyróżnione: każdy fragment tej samej długości jest równie prawdopodobny.

Rozkład **wykładniczy** opisuje czas oczekiwania na pierwsze zdarzenie w strumieniu zdarzeń Poissona: czas do awarii, czas między zgłoszeniami. Jest ciągłym odpowiednikiem geometrycznego i, jak on, nie ma pamięci; poświęcam mu osobną sekcję, bo wraca najczęściej z całej siódemki. Rozkład **normalny** powstaje z sumy wielu małych, niezależnych wpływów, z których żaden nie dominuje — stąd wzrost, błąd pomiaru, sumy drobnych zakłóceń. Suma niezależnych zmiennych normalnych jest znowu normalna, z sumą wartości oczekiwanych i wariancji.

Rozkład	Wartość oczekiwana	Wariancja
dwupunktowy p	p	$p(1 - p)$
dwumianowy n, p	np	$np(1 - p)$
geometryczny p	$1/p$	$(1 - p)/p^2$
Poissona λ	λ	λ
jednostajny $[a, b]$	$(a + b)/2$	$(b - a)^2/12$
wykładniczy λ	$1/\lambda$	$1/\lambda^2$
normalny μ, σ^2	μ	σ^2

! Poissona jest jedynym rozkładem w tabeli, w którym wartość oczekiwana równa się wariancji. Dla wykładniczego wariancja to kwadrat wartości oczekiwanej, więc odchylenie standardowe jest jej równe.

2.4 Rozkład wykładniczy i jego trzy własności

brak pamięci

Formalnie własność zapisuje się jako $P(X > s + t \mid X > t) = P(X > s)$ dla wszystkich $s, t > 0$. Praktycznie znaczy to, że urządzenie, które przepracowało już dwa lata, jest dokładnie tak samo dobre jak nowe — informacja o dotychczasowym czekaniu nie mówi nic o czekaniu, które zostało. Brzmi nienaturalnie, bo prawdziwe maszyny się zużywają — wykładniczy modeluje nie zużycie, lecz awarie przychodzące przypadkowo. Wśród rozkładów ciągłych jest jedynym bez pamięci, a wśród dyskretnych jedynym takim jest geometryczny.

stała funkcja hazardu

Funkcja hazardu opisuje chwilowe ryzyko zdarzenia pod warunkiem dotrwania do czasu t :

$$h(t) = \frac{f(t)}{1 - F(t)} = \lambda.$$

Mianownik $1 - F(t)$ to prawdopodobieństwo dotrwania, więc dzielenie przez niego jest warunkowaniem na „jeszcze działa”. Dla wykładniczego wychodzi stała — to ta sama myśl co brak pamięci, powiedziana językiem ryzyka.

jak liczy się rozkład minimum i maksimum

To rozumowanie jest proste, a odblokowuje całą klasę zadań. Minimum i maksimum liczy się nie przez gęstość, ale przez dystrybuantę, bo tylko ona pozwala przetłumaczyć zdanie o minimum na zdanie o wszystkich zmiennych naraz. Zdanie „minimum jest większe od x ” znaczy dokładnie to samo co „każda ze zmiennych jest większa od x ” – gdyby choć jedna była mniejsza, to ona byłaby minimum. Dla niezależnych zmiennych o wspólnej dystrybuancie F wolno więc pomnożyć prawdopodobieństwa i dostajemy $P(\min > x) = (1 - F(x))^n$. Symetrycznie „maksimum jest mniejsze od x ” znaczy „wszystkie są mniejsze od x ”, co daje $P(\max \leq x) = F(x)^n$.

Podstawmy pod ten schemat rozkład wykładniczy, dla którego $1 - F(x) = e^{-\lambda x}$. Dla niezależnych zmiennych o parametrach $\lambda_1, \dots, \lambda_n$ mnożenie ogonów daje

$$P(\min > x) = e^{-\lambda_1 x} \dots e^{-\lambda_n x} = e^{-(\lambda_1 + \dots + \lambda_n)x},$$

a to jest ogon rozkładu wykładniczego z parametrem będącym sumą parametrów. Minimum niezależnych zmiennych wykładniczych jest więc znowu wykładnicze.

- ▷ *Sens jest namacalny: jeśli układ ma n elementów psujących się niezależnie, to czas do pierwszej awarii całości jest wykładniczy, a średni czas oczekiwania to $1/(\lambda_1 + \dots + \lambda_n)$. O maksimum tak powiedzieć nie można – ono wykładnicze nie jest.*

2.5 Nierówności Markowa i Czebyszewa

nierówność Markowa

Dla zmiennej *nieujemnej* X i dowolnego $a > 0$ zachodzi $P(X \geq a) \leq EX/a$. Sens jest taki, że sama znajomość średniej ogranicza, jak często zmienna może być duża. Jeśli przeciętna pensja wynosi pięć tysięcy, to zarabiających powyżej pięćdziesięciu tysięcy nie może być więcej niż dziesięć procent – w przeciwnym razie oni sami wyciągnęliby średnią wyżej. Nieujemność jest niezbędna, bo inaczej wartości dodatnie byłyby kompensowane przez ujemne.

- ! Założenie $X \geq 0$ jest pierwszą rzeczą, którą autorzy testu usuwają – zdanie „dla dowolnej zmiennej losowej $P(X \geq a) \leq EX/a$ ” jest fałszywe. Zwróć też uwagę, że nierówność jest nietrywialna tylko dla $a > EX$; w przeciwnym razie prawa strona przekracza jedynkę i nic nie mówi.

nierówność Czebyszewa

Gdy znasz nie tylko średnią, ale i wariancję, możesz ograniczyć odchylenia w obie strony:

$$P(|X - EX| \geq t) \leq \frac{\text{Var } X}{t^2}.$$

Podstawiając $t = k\sigma$ dostajemy $1/k^2$: poza dwoma odchyleniami standardowymi leży najwyżej jedna czwarta masy, poza trzema najwyżej jedna dziewiąta. Nierówność wynika z Markowa zastosowanego do nieujemnej zmiennej $(X - EX)^2$, więc poza istnieniem wariancji nie potrzebuje żadnego założenia.

Skoro te granice są tak luźne, po co je stosować? Bo nie wymagają wiedzy o rozkładzie: dla normalnego poza trzema odchyleniami leży 0,3% masy, a Czebyszew dopuszcza aż 11% – ale obowiązuje dla *każdego* rozkładu o skończonej wariancji. Dla sum niezależnych zmiennych mocniejsza jest nierówność Chernoffa, w której ograniczenie maleje wykładniczo z odchyleniem.

- ▷ *Na teście działa to bardzo wygodnie: skoro nierówność podaje ograniczenie górne, każde słabsze, czyli*

większe, też jest prawdziwe. Jeżeli Czebyszew daje $1/4$, a stwierdzenie mówi „nie więcej niż $0,4$ ”, to jest prawdziwe, mimo że oszacowanie wygląda niepotrzebnie hojnie.

2.6 Centralne twierdzenie graniczne

co dokładnie mówi

Weźmy niezależne zmienne o tym samym rozkładzie, wartości oczekiwanej μ i skończonej wariancji σ^2 , i oznaczmy ich sumę przez S_n . Twierdzenie mówi, że po odjęciu wartości oczekiwanej i podzieleniu przez odchylenie standardowe sumy rozkład zbiega do standardowego normalnego:

$$\frac{S_n - n\mu}{\sigma\sqrt{n}} \rightarrow \mathcal{N}(0, 1).$$

Odejmujemy $n\mu$, bo wartość oczekiwana sumy n składników rośnie liniowo. Dzielimy przez $\sigma\sqrt{n}$, bo wariancja sumy niezależnych składników rośnie liniowo, a odchylenie standardowe jest jej pierwiastkiem. Rozrzut sumy rośnie więc wolniej niż sama suma i właśnie dlatego średnia z próby stabilizuje się przy dużym n .

Twierdzenie de Moivre’a i Laplace’a to ten sam wynik dla składników dwupunktowych, gdy suma ma rozkład dwumianowy: ponieważ $\mathbb{E}S_n = np$ i $\text{Var } S_n = np(1-p)$, dzielimy wtedy przez $\sqrt{np(1-p)}$. Przybliża się tak prawdopodobieństwa dwumianowe, gdy n jest duże, a p nie skrajnie małe; przy bardzo małym p lepszy jest Poisson.

czego twierdzenie nie mówi

Tu kryje się większość fałszywych stwierdzeń. Twierdzenie nie mówi, że składniki mają rozkład normalny ani że taki się staje — normalny jest tylko rozkład granicznej, *ustandaryzowanej* sumy. Nie mówi też nic o ogonach przy skończonej próbie: przybliżenie jest dobre w środku rozkładu, a im dalej w ogon, tym błąd większy, więc szacowanie bardzo małych prawdopodobieństw przy małym n jest zawodne.

2.7 Metoda największej wiarygodności

o co w niej chodzi

Masz dane i model z nieznanym parametrem. Metoda każe wybrać taką wartość parametru, przy której zebrane dane byłyby najbardziej prawdopodobne. Odwraca więc zwykły sposób myślenia: normalnie parametr jest ustalony, a dane losowe; tutaj dane są ustalone, a parametr traktujemy jak pokrętko, którym kręcimy, szukając najlepszego dopasowania. Przepis ma cztery kroki:

1. Zapisz funkcję wiarygodności, czyli prawdopodobieństwo lub gęstość zaobserwowanej próby jako funkcję parametru. Dla próby niezależnej jest to iloczyn $L(\theta) = \prod_{i=1}^n f(x_i; \theta)$.
2. Zlogarytmuj: $\ell(\theta) = \ln L(\theta) = \sum_i \ln f(x_i; \theta)$.
3. Zróżniczkuj po parametrze i przyrównaj do zera.
4. Sprawdź, że punkt jest maksimum, a nie minimum, i — co ważne — że maksimum nie leży na brzegu zakresu dopuszczalnych parametrów.

Logarytmuje się z dwóch powodów. Po pierwsze, logarytm zamienia iloczyn na sumę, a sumę różnic-

kuje się bez wzoru na pochodną iloczynu n czynników. Po drugie, jest funkcją rosnącą, więc *nie zmienia położenia maksimum* – punkt, w którym L jest największa, jest tym samym, w którym największa jest $\ln L$.

! Krok czwarty nie jest formalnością. Dla rozkładu jednostajnego na $(0, \theta)$ funkcja wiarygodności wynosi $1/\theta^n$ dla θ większych od wszystkich obserwacji i zero dla mniejszych, bo próba nie może wypaść poza przedziałem. Jest więc malejąca tam, gdzie jest niezerowa, i osiąga maksimum na brzegu: chcemy θ najmniejszego, ale nie na tyle małego, żeby wykluczyć zaobserwowane dane. Pochodna nigdzie się nie zeruje, a estymatorem jest maksimum z próby, $\hat{\theta} = \max(X_1, \dots, X_n)$.

2.8 Wartość p, poziom istotności i błędy dwóch rodzajów

czym jest wartość p

Wartość p to prawdopodobieństwo zobaczenia wyniku co najmniej tak skrajnego jak zaobserwowany, *przy założeniu, że hipoteza zerowa jest prawdziwa*. Cała treść siedzi w tym zastrzeżeniu: liczymy w świecie, w którym hipoteza zerowa obowiązuje, i sprawdzamy, jak dziwne byłyby w nim nasze dane. Sam próg, czyli poziom istotności α , ustala się z góry, przed zobaczeniem danych, bo dobieranie go po fakcie unieważnia całe rozumowanie.

czym wartość p nie jest: Wartość p *nie* jest prawdopodobieństwem, że hipoteza zerowa jest prawdziwa, ani że alternatywna jest fałszywa, ani prawdopodobieństwem popełnienia błędu. To znów podmiana kierunku warunkowania: wartość p to $P(\text{dane} \mid H_0)$, a nie $P(H_0 \mid \text{dane})$, a tej drugiej liczby nie da się policzyć bez podania, jak prawdopodobna była hipoteza zerowa przed zebraniem danych. Dodatkowo $p = 0,03$ nie mówi nic o wielkości efektu.

dwa rodzaje błędu i dlaczego grają przeciw sobie

Błąd pierwszego rodzaju to odrzucenie prawdziwej hipotezy zerowej, czyli fałszywy alarm; jego prawdopodobieństwo to właśnie α . Błąd drugiego rodzaju to przeoczenie istniejącego efektu, czyli nieodrzućenie hipotezy zerowej, choć jest fałszywa; jego prawdopodobieństwo oznacza się β , a liczbę $1 - \beta$ nazywa się mocą testu.

Zmniejszenie α , na przykład z 0,05 do 0,01, to wyższy wymóg dowodowy: wynik musi być *jeszcze* bardziej skrajny, więc obszar odrzucenia się zwęża. Ale ten sam zwężony obszar trudniej trafić także wtedy, gdy hipoteza zerowa jest naprawdę fałszywa – prawdziwe efekty przestają wystarczać. Rośnie więc β , a moc testu spada. Przy stałej liczności próby nie da się zmniejszyć obu błędów naraz; jedynym sposobem jest zwiększenie próby.

! Nieodrzućenie hipotezy zerowej nie jest jej dowodem – znaczy tylko, że dane nie dały dość mocnych przesłanek, żeby ją odrzucić. Zdanie „test nie wykazał różnicy, więc różnicy nie ma” jest zawsze fałszywe.

2.9 Nieobciążoność, zgodność i efektywność

trzy własności estymatora

Estymator **nieobciążony** ma wartość oczekiwaną równą szacowanemu parametrowi, $E\hat{\theta} = \theta$ — w jednej próbie prawie nigdy nie trafia, ale nie myli się systematycznie w jedną stronę. **Zgodny** przy rosnącej próbie zbiega do parametru; obciążony też może być zgodny, jeśli obciążenie znika przy rosnącym n . **Efektywny** ma najmniejszą wariancję wśród nieobciążonych.

skąd bierze się dzielenie przez mniej

W wariancji próbkowej sumę kwadratów $\sum_i (X_i - \bar{X})^2$ dzielimy przez $n - 1$, nie przez n . Przyczyna: odchylenia liczymy od średniej z próby, dopasowanej do tych samych danych — punktu, wokół którego suma kwadratów jest *najmniejsza możliwa*. Odchylenia wychodzą więc systematycznie za małe: dzielenie przez n daje estymator obciążony w dół, a $n - 1$ to koryguje.

ROZDZIAŁ 3

Algebra liniowa

algebra liniowa: Algebra jest na tym egzaminie działem o najwyższej przewidywalności: te same motywy wracają rok po roku, bo bardzo łatwo układa się z nich stwierdzenia prawda/fałsz. Autorzy testu nie proszą o żmudne rachunki — proszą o rozstrzygnięcie, czy zbiór jest podprzestrzenią, czy macierz jest diagonalizowalna, czy podany wektor jest własny i czy forma jest dodatnio określona. Wszystkie te pytania łączy jedno: da się je rozstrzygnąć jednym spostrzeżeniem albo jednym mnożeniem, jeśli tylko wiadomo, na co patrzeć. Dlatego najwięcej miejsca dostają tu wartości własne z obiema krotnościami — to rdzeń, z którego wyrasta i diagonalizowalność, i twierdzenie spektralne.

3.1 Liczby zespolone w postaci wykładniczej

po co zmieniać postać

Ta sama liczba zespolona ma dwa opisy. W postaci algebraicznej $z = a + bi$ łatwo się dodaje, ale mnożenie jest mozolne, a wysoka potęga praktycznie niewykonalna. W postaci wykładniczej $z = r e^{i\varphi}$, gdzie $r = |z|$, jest odwrotnie: dodawanie robi się niewygodne, ale mnożenie i potęgowanie stają się trywialne. Mechanizm jest jeden: *przy mnożeniu moduły się mnożą, a argumenty dodają*. Stąd

$$z_1 z_2 = r_1 r_2 e^{i(\varphi_1 + \varphi_2)}, \quad z^n = r^n e^{in\varphi}.$$

Potęgowanie jest szczególnym przypadkiem mnożenia liczby przez samą siebie, więc argument po prostu mnoży się przez n . Geometrycznie mnożenie przez z to obrót o kąt φ złożony z rozciągnięciem w skali r .

wysoka potęga i redukcja argumentu

Najczęstsze zadanie z tego tematu wygląda groźnie, a jest krótkie. Weźmy $z = 1 + i\sqrt{3}$ i policzmy z^{40} . Moduł to $\sqrt{1+3} = 2$, argument to $\pi/3$, więc $z^{40} = 2^{40} e^{i40\pi/3}$. Kąt $40\pi/3$ jest bezużyteczny, bo to ponad trzynaście π — ale argument jest określony tylko z dokładnością do pełnych obrotów, więc wolno odjąć dowolną wielokrotność 2π . Odejmujemy 12π , czyli sześć pełnych obrotów, i zostaje $40\pi/3 - 12\pi = 4\pi/3$. Ten kąt rozpoznajemy już z tablicy wartości, więc $z^{40} = 2^{40}(-\frac{1}{2} - \frac{\sqrt{3}}{2}i) = -2^{39} - 2^{39}\sqrt{3}i$. Całe zadanie to jedno odejmowanie — i warto odejmować wielokrotności 2π , bo odjęcie nieparzystej wielokrotności π odwraca znak wyniku.

kiedy potęga wraca do jedynek

Drugi ulubiony motyw brzmi: dla jakich n zachodzi $z^n = 1$? Jeśli $z = e^{2\pi i p/q}$, przy czym ułamek p/q jest nieskracalny, to $z^n = e^{2\pi i np/q}$ jest jedynką dokładnie wtedy, gdy np/q jest liczbą całkowitą. Skoro p i q nie mają wspólnego dzielnika, cała robota spada na n : $z^n = 1$ wtedy i tylko wtedy, gdy q dzieli n . Liczbę q nazywa się rzędem z : to najmniejsza dodatnia potęga dająca jedynkę, a wszystkie pozostałe są jej wielokrotnościami.

! Dwie pułapki, obie bardzo częste. Pierwsza: nieskrócenie ułamka. Liczba $e^{2\pi i \cdot 4/6}$ to po skróceniu $e^{2\pi i \cdot 2/3}$, więc jej rząd wynosi 3, a nie 6. Druga: wniosek, że n musi być podzielne przez $2q$ — dwójka z 2π została już zużyta w wykładniku i nie ma prawa pojawić się w warunku podzielności.

pierwiastki jako wielokąt foremny

Równanie $z^n = w$ ma dokładnie n rozwiązań i wszystkie leżą na jednym okręgu o promieniu $\sqrt[n]{|w|}$. Ich argumenty to $(\arg w + 2k\pi)/n$ dla $k = 0, 1, \dots, n-1$, czyli kolejne pierwiastki są oddalone o stały kąt $2\pi/n$ — tworzą więc wierzchołki n -kąta foremnego wpisanego w ten okrąg. Wystarczy znaleźć *jeden* pierwiastek, a pozostałe dostaje się obrotem.

To od razu daje sposób na pytanie, ile pierwiastków ma argument w zadanym przedziale: skoro wierzchołki leżą równomiernie ze skokiem $2\pi/n$, to w przedziale długości L mieści się ich około $Ln/2\pi$. Dla $z^{12} = 1$ skok wynosi 30° , a argumenty pierwiastków to $0^\circ, 30^\circ, \dots, 330^\circ$; w przedziale otwartym od 0° do 90° są zatem dwa.

! Gdyby ten sam przedział był domknięty, odpowiedzią byłoby cztery, bo weszłyby jeszcze pierwiastki dla 0° i 90° . Sprawdzaj osobno oba końce, zamiast dzielić długość przedziału przez skok.

3.2 Jak rozstrzygnąć, czy zbiór jest podprzestrzenią

co właściwie trzeba sprawdzić

Formalnie podprzestrzeń to niepusty podzbiór zamknięty na dodawanie i na mnożenie przez skalar, ale z definicji sprawdza to mało kto — w zadaniach egzaminacyjnych wystarcza krótsza droga:

1. Sprawdź, czy wektor zerowy należy do zbioru. Jeśli nie — koniec, to nie jest podprzestrzeń.
2. Sprawdź, czy warunki są *jednorodnymi równaniami liniowymi*: każdy składnik jest współrzędną pomnożoną przez stałą, a po prawej stronie stoi zero.
3. Jeśli tak — jest to podprzestrzeń. Jeśli nie, przyjrzyj się bliżej.

dlaczego wyraz wolny wszystko psuje

Warunek w rodzaju $x + y = 1$ dyskwalifikuje zbiór z dwóch niezależnych powodów. Po pierwsze, wektor zerowy go nie spełnia, a podprzestrzeń musi zawierać zero, bo mnożenie jej elementu przez skalar 0 nie może z niej wyprowadzać. Po drugie, dodanie dwóch rozwiązań dawałoby po prawej stronie 2, a nie 1. Geometrycznie to prosta, która nie przechodzi przez początek układu.

Odwrotna strona tej samej monety jest bardzo wygodna: zbiór rozwiązań układu jednorodnego $Ax = 0$ jest podprzestrzenią zawsze, bo z $Ax = 0$ i $Ay = 0$ wynika $A(\alpha x + \beta y) = \alpha Ax + \beta Ay = 0$. Nie ma znaczenia, ile jest równań ani czy są zależne — wystarczy, że po prawej stronie stoją zera, i nie trzeba sprawdzać niczego więcej.

pułapka z warunkiem kwadratowym

Najciekawsze stwierdzenia z tego tematu grają na tym, że wygląd warunku nie zdradza jego natury. Rozważmy w $\mathbb{R}^3$ zbiór opisany równaniem $x^2 + y^2 = 0$. Wygląda kwadratowo, więc odruch podpowiada „to nie może być podprzestrzeń” — i odruch jest błędny. Nad liczbami rzeczywistymi suma dwóch kwadratów jest zerem tylko wtedy, gdy oba składniki są zerami, więc ten warunek jest *równoważny* układowi $x = 0, y = 0$. Opisany zbiór to zwyczajna oś z , czyli podprzestrzeń wymiaru 1. Kwadraty były przebraniem.

Zupełnie inaczej zachowuje się bardzo podobnie wyglądający warunek $x^2 + y^2 = z^2$. Tego nie da się rozpisać na równania liniowe — to stożek i podprzestrzeń nie jest. Najkrótszym uzasadnieniem jest kontrprzykład: wektory $(1, 0, 1)$ i $(0, 1, 1)$ oba spełniają równanie, bo $1 + 0 = 1$ i $0 + 1 = 1$, ale ich suma $(1, 1, 2)$ daje po lewej stronie $1 + 1 = 2$, a po prawej $2^2 = 4$. Suma wypadła poza zbiór, więc zamkniętości na dodawanie nie ma. Zauważ przy tym, że zero do stożka należy i mnożenie przez skalar z niego nie wyprowadza — stożek przewraca się dopiero na dodawaniu.

3.3 Jądro i obraz przekształcenia liniowego

co znaczą te dwie przestrzenie

Przekształcenie liniowe $F : V \rightarrow W$ rozumie się przez dwie związane z nim podprzestrzenie. Jądro $\ker F = \{v : F(v) = 0\}$ to wektory, które przekształcenie *zgniata* do zera, czyli informacja bezpowrotnie tracona. Obraz $\operatorname{im} F = \{F(v)\}$ to wektory, które F potrafi *wyprodukować*. Jądro mieszka w przestrzeni wyjściowej, obraz w docelowej — mieszanie tych adresów jest najczęstszym źródłem pomyłki. W języku macierzy $\dim \operatorname{im} F$ to rząd macierzy.

twierdzenie o wymiarze

Te dwie liczby nie są niezależne:

$$\dim \ker F + \dim \operatorname{im} F = \dim V.$$

Intuicja jest prosta: każdy wymiar przestrzeni wyjściowej ma tylko dwa możliwe losy — albo ginie w jądrze, albo przeżywa i pojawia się w obrazie. Nic nie może zdarzyć się jednocześnie i nic nie ginie bez śladu, więc suma musi dawać $\dim V$, niezależnie od tego, jak dziwne jest samo F . Im więcej przekształcenie *zgniata*, tym mniej produkuje.

jak poznać przekształcenie

Przekształcenie liniowe jest w pełni określone przez to, co robi z wektorami bazy: skoro każdy wektor jest kombinacją bazowych, to jego obraz jest tą samą kombinacją obrazów bazy. Wystarczy więc zadziałać F na kolejnych wektorach bazy i zapisać wyniki jako kolumny macierzy.

Ulubioną areną tego egzaminu jest przestrzeń wielomianów stopnia co najwyżej 2 z bazą $1, x, x^2$. Weźmy $F(p) = p - x p'$. Dla $p = 1$ mamy $p' = 0$, więc $F(1) = 1$; dla $p = x$ wychodzi $F(x) = x - x = 0$; dla $p = x^2$ wychodzi $F(x^2) = x^2 - 2x^2 = -x^2$. Trzy rachunki i przekształcenie jest rozpoznane — w bazie $1, x, x^2$ jego macierz to

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{pmatrix}.$$

Teraz widać wszystko naraz. Środkowa kolumna jest zerowa, więc $\ker F = \operatorname{lin}(x)$ ma wymiar 1. Rząd wynosi 2, więc obraz ma wymiar 2; konkretnie $\operatorname{im} F = \operatorname{lin}(1, x^2)$, bo ogólny wynik to $F(a + bx + cx^2) = a - cx^2$. Suma wymiarów daje $1 + 2 = 3$, czyli dokładnie wymiar przestrzeni wyjściowej — twierdzenie się zgadza i posłużyło jako darmowa kontrola rachunku.

czy podany układ jest bazą obrazu

To pytanie ma trzy niezależne warunki: każdy wektor musi *należać* do obrazu, wektory muszą być *liniowo niezależne*, a ich *liczba* musi zgadzać się z wymiarem obrazu. Każdy z nich bywa psuty osobno, więc każdy

sprawdzaj osobno. W naszym przykładzie obraz to $\text{lin}(1, x^2)$ o wymiarze 2. Układ $\{1, x\}$ bazą obrazu nie jest, mimo dwóch niezależnych wielomianów — przewraca się na pierwszym warunku, bo wynik $a - cx^2$ nigdy nie ma składnika z x . Układ $\{1 + x^2, 1 - x^2\}$ już bazą jest. Baza obrazu nie jest więc jedyna — ale bez sprawdzenia przynależności dałoby się „udowodnić”, że jest nią dowolna baza przestrzeni docelowej.

3.4 Wartości i wektory własne

wektor własny

Przekształcenie na ogół obraca wektory i zmienia ich długość. Czasem jednak trafia się kierunek, którego nie rusza — wektor wychodzi przemnożony przez liczbę, ale leży na tej samej prostej. Takie wektory nazywamy własnymi, a współczynnik skalowania λ — wartością własną:

$$Av = \lambda v, \quad v \neq 0.$$

Wymóg $v \neq 0$ nie jest formalnością: wektor zerowy spełniałby to równanie dla każdej λ i pojęcie straciłoby sens. Sama λ może natomiast być zerem — to znaczy, że A zgniata jakiś kierunek do zera, czyli jest nieodwracalna. Cała diagonalizacja polega zaś na przejściu do bazy złożonej z kierunków własnych: w takiej bazie przekształcenie przestaje mieszać współrzędne i skaluje każdą z nich osobno.

skąd bierze się wielomian charakterystyczny

Przepiszmy warunek jako $(A - \lambda I)v = 0$. Szukamy więc λ , dla której macierz $A - \lambda I$ ma *niezerowe* jądro. A macierz kwadratowa ma niezerowe jądro dokładnie wtedy, gdy jest nieodwracalna, czyli gdy jej wyznacznik jest zerem — stąd warunek $\det(A - \lambda I) = 0$. Lewa strona jest wielomianem stopnia n zmiennej λ i nazywa się wielomianem charakterystycznym; wartości własne to jego pierwiastki. Nie ma tu magii — to tylko przetłumaczenie pytania o kierunki na pytanie o zerowanie się wyznacznika.

dwie krotności

Tu rozstrzyga się większość stwierdzeń z tego działu. **Krotność algebraiczna** wartości własnej to jej krotność jako pierwiastka wielomianu charakterystycznego. **Krotność geometryczna** to wymiar przestrzeni własnej, czyli

$$\dim \ker(A - \lambda I) = n - \text{rank}(A - \lambda I).$$

Krotność geometryczną liczy się zatem z *rzędu*: podstawia się konkretną wartość własną, sprowadza macierz do postaci schodkowej i odejmuje rząd od rozmiaru. Sens tej liczby jest bardzo konkretny — to liczba niezależnych kierunków własnych, jakie ta jedna wartość własna dostarcza. Zawsze zachodzi $1 \leq \text{krotność geometryczna} \leq \text{krotność algebraiczna}$, i tylko w tę stronę: wartość własna nie może dać więcej niezależnych kierunków, niż zapowiada wielomian charakterystyczny, ale może dać mniej — i w tym niedoborze kryje się cała trudność.

kryterium diagonalizowalności: Macierz jest diagonalizowalna wtedy i tylko wtedy, gdy dla *każdej* wartości własnej krotność geometryczna równa się algebraicznej. Wystarczy jedna wartość własna z niedoborem i diagonalizowalności nie ma. Równoważnie: krotności geometryczne muszą zsumować się do n , bo tyle niezależnych wektorów własnych potrzeba na bazę. Wniosek praktyczny — jeśli macierz $n \times n$ ma n różnych wartości własnych, jest diagonalizowalna automatycznie; rzędy warto liczyć tylko przy wartościach powtórzonych.

przykład macierzy, która się nie poddaje

Klasykiem jest $A = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$. Wielomian charakterystyczny to $(2 - \lambda)^2$, więc jedyna wartość własna $\lambda = 2$ ma krotność algebraiczną 2. Ale $A - 2I = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ ma rząd 1, więc jądro ma wymiar $2 - 1 = 1$ — krotność geometryczna jest mniejsza i macierz nie jest diagonalizowalna. Po ludzku: cała przestrzeń własna to jedna prosta rozpięta przez $(1, 0)$, a z jednego kierunku nie zbuduje się bazy płaszczyzny. Dla kontrastu macierz $2I$ ma *ten sam* wielomian charakterystyczny, a jest diagonalna, bo jej przestrzeń własna to cała płaszczyzna. Sam wielomian charakterystyczny o diagonalizowalności więc nie rozstrzyga — trzeba policzyć rząd.

! Powtórzona wartość własna nie oznacza automatycznie braku diagonalizowalności — macierz identycznościowa ma jedną wartość własną o pełnej krotności i jest diagonalna. Nie ma też związku z odwracalnością w żadną stronę: przykład powyżej jest odwracalny i niediagonalizowalny, a macierz zerowa dokładnie odwrotnie.

Dla macierzy niediagonalizowalnych istnieje postać zastępcza — postać Jordana, czyli macierz blisko diagonalna, z jedynkami nad przekątną w miejscach odpowiadających brakującym wektorom własnym; warto wiedzieć, że istnieje, ale konstrukcja jej bazy wykracza poza te notatki.

dwie sztuczki oszczędzające czas

Pierwsza dotyczy stwierdzeń w rodzaju „wektor $(1, 2, 1)$ jest wektorem własnym macierzy A ”. Nie trzeba do tego wielomianu charakterystycznego — wystarczy wykonać jedno mnożenie Av i sprawdzić, czy wynik jest krotnością v . Jeśli tak, współczynnik proporcjonalności jest szukaną wartością własną; jeśli nie, stwierdzenie jest fałszywe.

Druga to kontrola wyniku. Ślad macierzy, czyli suma elementów na przekątnej, oraz wyznacznik są związane z wartościami własnymi:

$$\operatorname{tr} A = \sum_i \lambda_i, \quad \det A = \prod_i \lambda_i,$$

przy czym każdą wartość własną liczymy tyle razy, ile wynosi jej krotność algebraiczna. Jeśli wyszły Ci wartości 1, 2, 3, a ślad wynosi 5, to gdzieś jest błąd i wiesz o tym natychmiast. Dla macierzy 2×2 te dwie równości bywają nawet szybszą drogą do wartości własnych niż wielomian charakterystyczny.

3.5 Wyznacznik i jego własności

co wyznacznik mierzy

Wyznacznik to liczba mówiąca, *ile razy* przekształcenie zmienia objętość: sześcian jednostkowy przepuszczony przez A staje się równoległościanem o objętości $|\det A|$. Znak niesie osobną informację — czy przekształcenie zachowuje orientację przestrzeni, czy ją odwraca, tak jak odbicie w lustrze zamienia lewą rękę na prawą. Wyznacznik zero znaczy, że przestrzeń została spłaszczona do czegoś cieńszego.

Ta jedna intuicja tłumaczy większość potrzebnych własności. Skoro wyznacznik jest współczynnikiem skalowania objętości, a złożenie przekształceń skaluje ją dwa razy po kolei, to $\det(AB) = \det A \cdot \det B$. Z tego natychmiast wynika wzór na macierz odwrotną: z $AA^{-1} = I$ i $\det I = 1$ dostajemy $\det(A^{-1}) = 1/\det A$. Widać też, dlaczego macierz o wyznaczniku zero odwrotnej mieć nie może — nie da się odzyskać objętości zgniecionej do zera. Transponowanie wyznacznika nie zmienia, czyli $\det A^T = \det A$, a stąd wniosek, że każda własność sformułowana dla wierszy obowiązuje równie dobrze dla kolumn. Wreszcie zdania „wyznacznik jest niezerowy”, „macierz jest odwracalna”, „rząd wynosi n ” i „układ $Ax = 0$ ma tylko rozwiązanie zerowe” znaczą dokładnie to samo — stwierdzenia egzaminacyjne krążą właśnie po tej liście.

mnożenie macierzy przez liczbę: Dla macierzy $n \times n$ zachodzi

$$\det(cA) = c^n \det A.$$

Wykładnik n bierze się stąd, że mnożąc całą macierz przez c , mnożymy przez c *każdy* z jej n wierszy, a wyznacznik jest liniowy względem każdego wiersza osobno — każdy wiersz wnosi więc jeden czynnik c . Geometrycznie: rozciągnięcie bryły c -krotnie w n kierunkach zwiększa objętość c^n razy.

! Napisanie $\det(cA) = c \det A$ to prawdopodobnie najczęstszy błąd w całym dziale. Sprawdź na najprostszym przykładzie: dla macierzy 3×3 mamy $\det(2I) = 2 \cdot 2 \cdot 2 = 8$, a nie 2, więc przy $\det A = 5$ wychodzi $\det(2A) = 40$.

3.6 Macierze symetryczne i określoność formy

dlaczego symetria tak wiele zmienia

Macierz symetryczna, czyli spełniająca $A^T = A$, jest najprzyjaźniejszym obiektem w tym dziale. Twierdzenie spektralne mówi, że rzeczywista macierz symetryczna ma wyłącznie *rzeczywiste* wartości własne oraz że zawsze daje się zdiagonalizować, i to w bazie *ortonormalnej*: $A = Q\Lambda Q^T$, gdzie Λ jest diagonalna, a kolumny Q to parami prostopadłe wektory własne o długości 1. Znikają więc naraz dwa kłopoty z poprzedniej sekcji: nie trzeba bać się wartości zespolonych ani sprawdzać krotności, bo dla macierzy symetrycznej krotność geometryczna zawsze równa się algebraicznej.

określoność formy kwadratowej

Formę kwadratową zapisuje się jako $q(x) = x^T A x$ z macierzą symetryczną A , a pytanie o określoność jest pytaniem o znak tej liczby: czy q jest dodatnie dla wszystkich niezerowych x , ujemne dla wszystkich, czy raz takie, raz takie. Twierdzenie spektralne odpowiada natychmiast, bo po przejściu do bazy własnej forma traci składniki mieszane:

$$q = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \dots + \lambda_n y_n^2.$$

Kwadraty są nieujemne, więc o znaku decydują wyłącznie współczynniki. Forma jest dodatnio określona dokładnie wtedy, gdy *wszystkie* wartości własne są dodatnie, i ujemnie określona, gdy wszystkie są ujemne. Gdy występują oba znaki, forma jest nieokreślona: na jednym kierunku rośnie, na innym maleje. Zero daje przypadek półokreślony — forma nie zmienia znaku, ale znika na całym kierunku.

kryterium Sylwestera

Liczenie wartości własnych bywa kosztowne, więc jest i szybsza droga, oparta na minorach głównych wiodących — wyznacznikach lewych górnych bloków 1×1 , 2×2 i tak dalej, aż do całej macierzy. Oznaczmy je $D_1, \dots, D_n$. Forma jest dodatnio określona wtedy i tylko wtedy, gdy wszystkie te minory są dodatnie, a ujemnie określona wtedy i tylko wtedy, gdy ich znaki są naprzemienne, poczynając od minusa: $D_1 < 0$, $D_2 > 0$, $D_3 < 0$ i tak dalej. Drugiego warunku nie trzeba nawet pamiętać — q jest ujemnie określona dokładnie wtedy, gdy $-q$ jest dodatnio określona, więc wystarczy zbadać macierz $-A$ pierwszym kryterium.

! Kryterium minorów wiodących działa tylko dla określoności ścisłej. Dla półokreśloności zamiana nierówności ostrych na nieostre *nie* jest poprawna: macierz $\text{diag}(0, -1)$ ma $D_1 = 0$ i $D_2 = 0$, czyli oba minory nieujemne, a forma $q = -y_2^2$ jest ujemnie półokreślona, nie dodatnio. Przy półokreśloności trzeba badać wszystkie minory główne, nie tylko wiodące — albo po prostu policzyć znaki wartości

własnych.

po co to w uczeniu maszynowym

Ten temat nie jest ozdobą programu: macierze kowariancji są symetryczne i dodatnio półokreślone, hessian funkcji kosztu decyduje swoją określonością o tym, czy punkt krytyczny jest minimum, a rozkład według wartości singularnych i analiza składowych głównych to bezpośrednie zastosowania twierdzenia spektralnego.

ROZDZIAŁ 4

Matematyka dyskretna

matematyka dyskretna: Ten dział jest nietypowo opłacalny, bo jedna umiejętność — porównywanie rzędów wzrostu — obsługuje zadania w trzech miejscach naraz: tutaj, w analizie i w algorytmach. Pozostałe tematy łączy to, że są twierdzeniami z wyraźnym założeniem, a fałszywe stwierdzenia powstają najczęściej przez usunięcie tego założenia: znika względna pierwszość w twierdzeniu Eulera, ograniczenie górne udaje dokładny rząd, warunek wystarczający udaje konieczny. Dlatego warto pamiętać nie tyle sam wzór, ile to, *co się psuje*, kiedy założenie odpadnie.

4.1 Rzędy wzrostu i notacje asymptotyczne

Cały temat sprowadza się do pytania, która z dwóch funkcji rośnie szybciej dla dużych n . Stałe mnożące i składniki niższego rzędu nie mają znaczenia. Notacje asymptotyczne są tylko językiem, w którym taką odpowiedź się zapisuje, a cały zysk bierze się ze zrozumienia, co która z nich mówi — a czego nie mówi.

duże O

Zapis $f(n) = \mathcal{O}(g(n))$ znaczy: od pewnego miejsca f nie przekracza g z dokładnością do stałej, czyli istnieją $c > 0$ i n_0 takie, że $f(n) \leq c g(n)$ dla $n \geq n_0$. To jest ograniczenie *górne* i nic ponad to: $\mathcal{O}$ nie twierdzi, że f rośnie tak szybko jak g , lecz że nie rośnie szybciej. A skoro jest to tylko sufit, wolno go zawiesić dowolnie wysoko i zdanie pozostanie prawdziwe.

! Stwierdzenie „algorytm działa w czasie $\mathcal{O}(n^2)$ ” jest *prawdziwe* dla sortowania przez scalanie, które działa w czasie $\Theta(n \log n)$. Odruch „przecież on jest szybszy, więc to fałsz” kosztuje na tym teście najwięcej punktów. Fałszywe byłoby dopiero $\Theta(n^2)$ albo $\Omega(n^2)$.

omega, theta i wersje ostre

Notacja Ω jest lustrzanym odbiciem: $f(n) = \Omega(g(n))$ znaczy, że od pewnego miejsca $f(n) \geq c g(n)$, czyli g jest *podłogą*. Notacja Θ to jedno i drugie naraz, więc tylko ona mówi, że funkcje rosną w tym samym tempie, i tylko ją da się obalić, pokazując, że jedna z nich jest wolniejsza.

Małe o i małe ω są wersjami ostrymi: $f = o(g)$ znaczy, że iloraz f/g dąży do zera, a $f = \omega(g)$ — że dąży do nieskończoności. Stąd konsekwencja, o którą łatwo się potknąć: $n^2 = \mathcal{O}(n^2)$ jest prawdą, ale $n^2 = o(n^2)$ fałszem, bo iloraz jest stale równy jeden.

▷ Roboczo: $\mathcal{O}$ to „ $\leq$ ”, Ω to „ $\geq$ ”, Θ to „ $=$ ”, a małe o i ω to ostre „ $<$ ” i „ $>$ ”.

wyrażenia piętrowe

Kiedy pojawia się ułamek z sumami w liczniku i mianowniku, nie trzeba niczego liczyć — wystarczy w każdej sumie zostawić *tylko składnik dominujący*, bo reszta i tak zniknie w granicy. Wyrażenie

$$\frac{3n^3 + 5n^2 \log n + 100n}{2n + \log n}$$

redukuje się do $n^3/n = n^2$, czyli jest $\Theta(n^2)$.

Osobno warto zapamiętać rzecz, na której ludzie myślą się pod presją czasu: *dzielenie przez $1/n$ to mnożenie przez n* , więc $\frac{n \log n}{1/n} = n^2 \log n$. Autorzy zadań ukrywają w mianowniku małe wyrażenie właśnie po to, żeby wynik urósł, a nie zmałał.

skala rzędów wzrostu

Poniższa kolejność wystarcza na cały egzamin i jest jedyną rzeczą w tym rozdziale, którą naprawdę opłaca się wykuć na pamięć; znak $\prec$ czytamy „rośnie istotnie wolniej niż”.

$$\begin{aligned} \log \log n &\prec \log n &\prec (\log n)^k &\prec \sqrt{n} &\prec n &\prec n \log n \\ &&&&&&&\prec n^2 &\prec n^3 &\prec \varphi^n &\prec 2^n &\prec 3^n &\prec n! &\prec n^n \end{aligned}$$

Warto pamiętać, skąd te rzędy pochodzą: $\log n$ to wyszukiwanie binarne i wysokość zrównoważonego drzewa, $n \log n$ to sortowanie przez porównania, 2^n to przegląd wszystkich podzbiorów, a $n!$ — wszystkich permutacji. Najbardziej zaskakuje trzecie miejsce: logarytm w dowolnie wysokiej potęgce, choćby $(\log n)^{100}$, przegrywa z *każdą* potęgą n^ϵ , choćby mikroskopijną.

wzór Stirlinga i dwa wnioski

Silnia jest niewygodna, bo nie jest ani potęgą, ani funkcją wykładniczą. Wzór Stirlinga mówi, jak wygląda z daleka:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

Wyprowadzenia znać nie trzeba, ale dwa wnioski — tak.

Pierwszy dostajemy, logarytmując: $\log(n!) = n \log n - n + \mathcal{O}(\log n)$, czyli po prostu $\log(n!) = \Theta(n \log n)$. To jest dolne ograniczenie na sortowanie przez porównania, więc wraca też w rozdziale o algorytmach.

Drugi jest jakościowy: $n!$ rośnie szybciej niż *każda* funkcja wykładnicza c^n , choćbyśmy wzięli olbrzymie c — iloraz zachowuje się bowiem jak $(n/(ec))^n$, a podstawa tej potęgi rośnie do nieskończoności. W c^n podstawa jest ustalona, a w $n!$ mnożymy przez coraz większe czynniki.

wykładnik z logarytmem

Wyrażenia w rodzaju $2^{3 \log_2 n}$ wyglądają groźnie, a są zwykłymi wielomianami — wystarczy przestawić kolejność potęgowania:

$$2^{c \log_2 n} = (2^{\log_2 n})^c = n^c,$$

więc $2^{3 \log_2 n}$ to po prostu n^3 .

! Kluczowe jest, gdzie siedzi n . Jeśli stoi w wykładniku pomnożone przez logarytm, wyrażenie jest wielomianem; jeśli stoi w wykładniku samo, jest wykładnicze. Podmiana jednego na drugie to gotowe fałszywe zdanie.

4.2 Rekurencje liniowe

metoda równania charakterystycznego

Dla rekurencji o stałych współczynnikach, na przykład $a_{n+2} = p a_{n+1} + q a_n$, szukamy rozwiązań postaci $a_n = x^n$. Po podstawieniu i podzieleniu przez x^n zostaje równanie charakterystyczne $x^2 = px + q$. Jeśli ma dwa różne pierwiastki, to każdy ciąg spełniający rekurencję ma postać $a_n = A x_1^n + B x_2^n$, a stałe wyznacza się z dwóch warunków początkowych. Gdy pierwiastek jest podwójny, jedna potęga nie wystarcza i drugim rozwiązaniem jest $n x^n$.

co naprawdę trzeba wiedzieć na teście

Prawie nigdy nie trzeba dokładnego wzoru. Wystarczy wiedzieć, *który pierwiastek jest największy co do modułu*, bo jego potęga zdominuje resztę.

Weźmy $a_{n+2} = a_{n+1} + 2a_n$. Równanie charakterystyczne to $x^2 - x - 2 = 0$, a jego pierwiastki to 2 i -1. Wobec tego $a_n = A \cdot 2^n + B \cdot (-1)^n$, gdzie drugi składnik jest wiecznie mały, bo waha się między $-B$ a B . Ciąg rośnie jak 2^n i to jest cała potrzebna informacja. Konkretnie, przy $a_0 = 0$, $a_1 = 1$ wychodzi $a_n = (2^n - (-1)^n)/3$, więc a_{90} to praktycznie $2^{90}/3$.

porównywanie bez liczenia

Czy a_{90} jest większe od $1,618^{90}$, a czy jest większe od 3^{44} ? Kalkulator nie jest potrzebny.

Pierwsze porównanie jest łatwe, bo wykładniki są takie same, a wtedy decyduje podstawa: $2 > 1,618$, więc 2^{90} wygrywa, i to nie o włos — iloraz wynosi $(2/1,618)^{90} \approx 1,236^{90}$, czyli mniej więcej sto milionów.

Drugie wymaga jednego ruchu: sprowadzamy oba wyrażenia do *wspólnego wykładnika*. Zauważamy, że $2^{88} = (2^2)^{44} = 4^{44}$, a $4^{44} > 3^{44}$, bo znów wykładniki są równe. Skoro $2^{90} = 4 \cdot 2^{88}$, to $a_{90} \approx \frac{4}{3} \cdot 4^{44}$, czyli wyraźnie więcej niż 3^{44} . Reguła jest więc taka: przy równych wykładnikach porównuj podstawy, a jeśli wykładniki się różnią, przerób jeden z nich na wielokrotność drugiego.

4.3 Twierdzenie o rekurencji uniwersalnej

To narzędzie do rekurencji postaci $T(n) = aT(n/b) + f(n)$, w których problem rozbijamy na a podproblemów rozmiaru n/b , a $f(n)$ jest kosztem podziału i scalenia. Cała treść twierdzenia to porównanie kosztu z dołu rekursji, czyli $n^{\log_b a}$, z kosztem z góry, czyli $f(n)$ — wygrywa ten, który dominuje.

trzy przypadki

Jeżeli $f(n)$ jest istotnie mniejsze od $n^{\log_b a}$ — mniejsze o czynnik wielomianowy, nie tylko o logarytm — to dominują liście i $T(n) = \Theta(n^{\log_b a})$. Tak jest w algorytmie Karatsuby, $T(n) = 3T(n/2) + \Theta(n)$, gdzie $n^{\log_2 3} \approx n^{1,585}$ przerasta koszt liniowy.

Jeżeli oba koszty są tego samego rzędu, praca rozkłada się równo na $\log_b n$ poziomów, więc wynik to $\Theta(n^{\log_b a} \log n)$. Wzorcowe jest sortowanie przez scalanie, $T(n) = 2T(n/2) + \Theta(n)$, gdzie $n^{\log_2 2} = n$ równoważy koszt scalania, co daje $\Theta(n \log n)$. Tu należy też wyszukiwanie binarne, $T(n) = T(n/2) + \Theta(1)$, z wynikiem $\Theta(\log n)$.

Jeżeli natomiast $f(n)$ jest istotnie większe od $n^{\log_b a}$, dominuje górny poziom i $T(n) = \Theta(f(n))$; na przykład $T(n) = 2T(n/2) + \Theta(n^2)$ daje $\Theta(n^2)$.

! Twierdzenie milczy, gdy różnica między $f(n)$ a $n^{\log_b a}$ nie jest wielomianowa. Dla $T(n) = 2T(n/2) + n \log n$ iloraz wynosi $\log n$, co nie jest żadną potęgą n^ϵ , więc żaden przypadek nie zachodzi, a odpowiedź $\Theta(n \log^2 n)$ wymaga innej metody.

4.4 Liczby Fibonacciego i złoty podział

skąd bierze się wzór Bineta

Ciąg Fibonacciego, $F_{n+1} = F_n + F_{n-1}$, to zwykła rekurencja liniowa, więc obowiązuje procedura z poprzedniej sekcji. Równanie charakterystyczne to $x^2 = x + 1$, a jego pierwiastki to

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1,618, \quad \psi = \frac{1 - \sqrt{5}}{2} \approx -0,618.$$

Stałe wyznaczone z warunków $F_0 = 0, F_1 = 1$ dają wzór Bineta $F_n = (\varphi^n - \psi^n)/\sqrt{5}$.

Ciekawe jest to, co dzieje się z drugim składnikiem: ponieważ $|\psi| < 1$, potęga ψ^n szybko dąży do zera, więc F_n jest po prostu najbliższą liczbą całkowitą do $\varphi^n/\sqrt{5}$; ciąg rośnie wykładniczo z podstawą φ .

dlaczego to nie jest to samo co dwa do n

Podstawy różnią się o niecałe 0,4, co brzmi niewiele, ale przy potęgowaniu daje przepaść: φ^n to mniej więcej $2^{0,694n}$, więc dla $n = 100$ mówimy o różnicy rzędu 2^{30} , czyli ponad miliarda razy.

! Zdanie „ F_n rośnie jak 2^n ” jest fałszywe, choć oba ciągi są wykładnicze. Podobnie fałszywe jest, że drzewo AVL o wysokości h ma co najmniej 2^h węzłów — ma ich około φ^h , i właśnie dlatego gwarantowana wysokość drzewa AVL wynosi około $1,44 \log_2 n$, a nie $\log_2 n$.

4.5 Funkcje tworzące

cała idea w jednym zdaniu

Ciąg $a_0, a_1, a_2, \dots$ pakujemy w jeden obiekt

$$A(x) = \sum_{n \geq 0} a_n x^n = a_0 + a_1 x + a_2 x^2 + \dots,$$

i od tej chwili operacje na ciągu stają się operacjami na funkcji. Zmienna x jest tu wyłącznie wieszakiem na współczynniki — nie podstawiamy pod nią liczb i nie pytamy o zbieżność, bo interesuje nas tylko to, co stoi przy kolejnych potęgach. Warto tę myśl powiedzieć sobie wprost, bo bez niej wzory z tego tematu wyglądają jak magia, a z nią stają się zwykłym tłumaczeniem z języka ciągów na język algebry.

podstawowy słownik

Cztery pozycje wystarczą na większość zadań. Szereg geometryczny $\frac{1}{1-x} = 1 + x + x^2 + \dots$ jest funkcją tworzącą ciągu samych jedynek. Podstawiając cx zamiast x , dostajemy $\frac{1}{1-cx}$ dla ciągu c^n . Dalej, $\frac{1}{(1-x)^2}$ odpowiada ciągowi $n + 1$: mnożąc dwa szeregi samych jedynek, przy x^n zbieramy wszystkie sposoby zapisania $n = i + j$, a tych jest dokładnie $n + 1$. Wreszcie ze wzoru dwumianowego $(1 + x)^n = \sum_k \binom{n}{k} x^k$, więc wielomian $(1 + x)^n$ jest funkcją tworzącą całego wiersza trójkąta Pascala.

dlaczego to działa na rekurencje

Kluczowa obserwacja jest taka, że *mnożenie przez x przesuwą ciąg*: szereg $xA(x)$ ma przy x^n współczynnik a_{n-1} . Zależność rekurencyjna, która wiąże wyraz z poprzednimi, zamienia się więc w równanie algebraiczne wiążące $A(x)$ z $xA(x)$ i $x^2A(x)$.

Na Fibonaccim widać to najlepiej. Z $F_n = F_{n-1} + F_{n-2}$ oraz $F_0 = 0, F_1 = 1$ dostajemy $A(x) = xA(x) + x^2A(x) + x$, czyli $A(x)(1 - x - x^2) = x$, a stąd

$$A(x) = \frac{x}{1 - x - x^2}.$$

Mianownik rozkłada się na czynniki liniowe o pierwiastkach związanych ze znanymi już φ i ψ , a rozbitcie tego ułamka na sumę ułamków prostych daje po odczytaniu współczynników dokładnie wzór Bineta. To jest ta sama odpowiedź co z równania charakterystycznego, tylko dojechaliśmy do niej inną drogą.

splot, czyli po co mnożyć funkcje tworzące

Współczynnik przy x^n w iloczynie $A(x)B(x)$ wynosi

$$\sum_{k=0}^n a_k b_{n-k},$$

co nazywamy splotem ciągów. Kombinatorycznie znaczy to tyle: obiekt rozmiaru n sklejamy z dwóch części, biorąc coś rozmiaru k z pierwszej rodziny i coś rozmiaru $n - k$ z drugiej. Mnożenie funkcji tworzących odpowiada więc *składaniu obiektów z części*, i to jest właściwy powód, dla którego są one naturalnym narzędziem kombinatoryki: zdanie „obiekt składa się z tego i z tamtego” tłumaczy się wprost na iloczyn.

wykładnicze funkcje tworzące

Czasem wygodniej pakować ciąg z dzielnikiem silni:

$$\hat{A}(x) = \sum_{n \geq 0} a_n \frac{x^n}{n!}.$$

Zwykłych funkcji tworzących używamy do obiektów **nieetykietowanych**, gdzie liczy się sam rozmiar części, a wykładniczych do obiektów **etykietowanych**, gdzie elementy są rozróżnialne i trzeba jeszcze zdecydować, które z nich trafią do której części. Silnie w mianownikach robią dokładnie tę robotę: przy mnożeniu dwóch wykładniczych funkcji tworzących przy $x^n/n!$ pojawia się automatycznie czynnik $\binom{n}{k}$, czyli wybór etykiet dla pierwszej części.

Najprostszy przykład to $e^x = \sum x^n/n!$, czyli wykładnicza funkcja tworząca ciągu samych jedynek – odpowiednik $\frac{1}{1-x}$ z tamtego świata. Ładnym zastosowaniem są nieporządki, czyli permutacje bez punktu stałego: ich wykładnicza funkcja tworząca to $\frac{e^{-x}}{1-x}$, skąd $D_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$, a więc D_n jest mniej więcej równe $n!/e$.

- Na teście funkcje tworzące przydają się do dwóch rzeczy: rozstrzygnięcia, czy podana zamknięta postać ciągu jest właściwa, oraz szybkiego sprawdzenia tożsamości sumacyjnej, bo suma po k z iloczynem dwóch współczynników to prawie zawsze splot.

4.6 Indukcja matematyczna

schemat i po co jest baza

Dowód indukcyjny ma dwie części: *bazę*, czyli sprawdzenie tezy dla najmniejszego n , i *krok*, czyli wykazanie, że z prawdziwości dla n wynika prawdziwość dla $n + 1$. Obraz z kostkami domina oddaje, po co potrzebne są obie części: krok mówi tylko tyle, że każda kostka przewróci następną, a baza — że ktoś popchnął pierwszą.

Bez bazy da się „udowodnić” rzeczy fałszywe. Weźmy zdanie $n = n + 1$: krok przechodzi bez zarzutu, bo dodając jedynkę stronami, z $n = n + 1$ dostajemy $n + 1 = n + 2$. Nie ma tylko żadnego n , dla którego teza byłaby prawdziwa, więc cała konstrukcja wisi w powietrzu.

indukcja silna

W wersji silnej w kroku wolno założyć tezę dla *wszystkich* $m \leq n$, a nie tylko dla samego n . Potrzeba jej wtedy, gdy dowód sięga wstecz dalej niż o jeden wyraz albo na nieprzewidywalną odległość. Typowe sytuacje to rekurencje odwołujące się do dwóch poprzednich wyrazów oraz twierdzenie o rozkładzie na czynniki pierwsze, gdzie liczbę złożoną zapisujemy jako $n = ab$ i korzystamy z tezy dla a i b — a o nich wiemy tylko tyle, że są mniejsze od n .

! Najczęstszy błąd to krok indukcyjny działający dopiero od pewnego miejsca przy bazie postawionej za nisko. Nierówność $2^n > n^2$ jest fałszywa dla $n = 2, 3, 4$, a prawdziwa od $n = 5$; sam krok przechodzi dla $n \geq 3$, więc dowód jest poprawny wyłącznie z bazą w $n = 5$. Zaczęcie od $n = 1$ daje łańcuch, który urywa się natychmiast.

4.7 Współczynniki dwumianowe i trójkąt Pascala

rekurencja Pascala i skąd się bierze

Poprawna postać to

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1},$$

a najlepiej pamiętać ją przez rozumowanie, nie przez kształt wzoru. Wybieramy k -elementowy podzbiór ze zbioru n -elementowego i patrzymy na jeden ustalony element. Albo go *nie bierzemy* — wtedy całe k elementów pochodzi z pozostałych $n - 1$, co daje $\binom{n-1}{k}$ — albo go *berzemy*, i wtedy dobieramy jeszcze $k - 1$ elementów z tych samych $n - 1$. Przypadki wykluczają się i wyczerpują wszystkie możliwości. Zwróć uwagę, że w obu składnikach górny wskaźnik to $n - 1$: *oba* razy schodzimy o jeden.

! Wariant $\binom{n-1}{k} + \binom{n}{k-1}$ jest *fałszywy*, ale trzeba go obalać ostrożnie, bo dla najmniejszych k bywa przypadkiem prawdziwy. Dla $k = 1$ mamy $\binom{n-1}{1} + \binom{n}{0} = (n - 1) + 1 = n = \binom{n}{1}$, czyli zgadza się zawsze, i dla $k = 0$ też. Dopiero od $k = 2$ pojawia się błąd, i to dokładnie o jeden.

Kontrprzykład trzeba więc dobrać świadomie. Najmniejszy sensowny to $n = 3, k = 2$: poprawnie $\binom{3}{2} = 3$, a fałszywy wariant daje $\binom{2}{2} + \binom{3}{1} = 1 + 3 = 4$.

tożsamość ze średnią wielkością podzbioru

Warto znać jeszcze jedną tożsamość, bo pojawia się w zadaniach o sumach:

$$\sum_{i=0}^n i \binom{n}{i} = n 2^{n-1}.$$

Uzasadnienie słowne jest łatwiejsze niż rachunek. Obie strony liczą tę samą rzecz: liczbę par złożonych z podzbioru i wyróżnionego elementu *w tym podzbiorze*. Lewa strona wybiera najpierw podzbiór, a potem lidera wewnątrz niego. Prawa robi to odwrotnie: najpierw wskazuje lidera spośród n osób, a potem dowolnie decyduje o pozostałych $n - 1$ elementach, czyli na 2^{n-1} sposobów.

4.8 Podstawowe schematy zliczania

cztery przypadki

Wszystko zależy od dwóch pytań: czy *kolejność* wybranych elementów ma znaczenie i czy elementy wolno *powtarzać*.

Jeśli kolejność ma znaczenie i powtórzenia są zabronione, wybieramy k elementów z n i ustawiamy je w ciąg – pierwszy na n sposobów, drugi na $n-1$ i tak dalej, co daje $n(n-1) \cdots (n-k+1) = \frac{n!}{(n-k)!}$. Jeśli kolejność ma znaczenie, ale powtórzenia są dozwolone, każde z k miejsc obsadzamy niezależnie i wychodzi n^k .

Jeśli kolejność nie ma znaczenia i nie ma powtórzeń, to zwykły podzbiór, czyli $\binom{n}{k}$ – dzielimy poprzedni wynik przez $k!$, bo każdy podzbiór policzyliśmy tyle razy, na ile sposobów dało się go ustawić. Zostają kombinacje z powtórzeniami, gdzie kolejność nie gra roli, ale ten sam element wolno wziąć kilka razy; tych jest

$$\binom{n+k-1}{k}.$$

Ostatni wzór najłatwiej zrozumieć obrazkiem z gwiazdkami i przegrodami: wybór sprowadza się do rozdzielenia k jednakowych kulek do n pudełek, a taki układ opisuje ciąg k kulek i $n-1$ przegród.

zasada włączeń i wyłączeń

Dla dwóch zbiorów mamy $|A \cup B| = |A| + |B| - |A \cap B|$, a w wersji ogólnej sumujemy pojedyncze zbiory, odejmujemy wszystkie przecięcia par, dodajemy przecięcia trójek i tak dalej, ze znakiem zmieniającym się na przemian.

Naprzemiennosc znaków nie jest kaprysem, tylko poprawianiem poprawek. Element należący do dokładnie dwóch zbiorów zostaje w pierwszej sumie policzony dwa razy, więc odjęcie przecięć par sprowadza go do jednego. Element należący do trzech zbiorów jest liczony trzy razy, a potem odejmowany trzy razy, bo leży w trzech parach – przez co znika zupełnie i trzeba go dodać z powrotem. Każde kolejne piętro naprawia to, co zepsuło poprzednie.

liczby Catalana

Ciąg 1, 1, 2, 5, 14, 42, ... zadany wzorem

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

warto rozpoznawać po modelach, które zlicza. Najbardziej znany to poprawne układy n par nawiasów – $C_3 = 5$ odpowiada pięciu sensownym wyrażeniom z trzema parami. Drugi to drzewa binarne o n węzłach,

a trzeci to podziały wypukłego wielokąta o $n + 2$ bokach na trójkąty przekątnymi. Wszystkie trzy opisują tę samą rzecz: rozbicie struktury na lewą i prawą część, co w języku poprzedniej sekcji jest po prostu splotem $C_{n+1} = \sum_k C_k C_{n-k}$.

4.9 Zasada szufladkowa Dirichleta

sformułowanie i wersja ogólna

W wersji podstawowej zasada mówi rzecz oczywistą: jeśli n obiektów rozmieszczamy w k szufladkach i $n > k$, to w którejś szufladce leżą co najmniej dwa obiekty. W wersji ogólnej, przy n obiektach i k szufladkach, w którejś szufladce jest co najmniej $\lceil n/k \rceil$ obiektów — bo gdyby wszędzie było mniej, suma nie uzbierałaby się do n . Zasada nie wskazuje przy tym, która szufladka jest przepełniona: daje dowody istnienia bez podania przykładu.

siła leży w wyborze szufladek

Samo sformułowanie jest banalne, a cała trudność zadań polega na wymyśleniu, co ma być szufladką. Dobry przykład: wybieramy $n + 1$ liczb ze zbioru $\{1, 2, \dots, 2n\}$ i chcemy pokazać, że któraś z nich dzieli inną. Za szufladkę bierzemy *nieparzysty dzielnik* — każdą liczbę zapisujemy jako $2^a \cdot m$ z nieparzystym m . Takich m w zakresie do $2n$ jest dokładnie n , a liczb mamy $n + 1$, więc dwie muszą mieć tę samą część nieparzystą. Wtedy jedna z nich to $2^a m$, druga $2^b m$, a ta o mniejszym wykładniku dzieli tę o większym.

Nic tu nie liczyliśmy — cały dowód sprowadził się do trafnego pomysłu, czym mają być szufladki. Zadania z tej zasady rozpoznaje się po tym, że pytają o istnienie pary obiektów w jakiejś relacji przy podejrzeniu okrągłej liczbie danych.

4.10 Arytmetyka modularna

po co są twierdzenia Fermata i Eulera

Oba służą jednemu celowi: redukcji wysokich wykładników. Małe twierdzenie Fermata mówi, że jeśli p jest liczbą pierwszą i a nie dzieli się przez p , to $a^{p-1} \equiv 1 \pmod{p}$. Twierdzenie Eulera uogólnia to na dowolny moduł: jeśli $\text{NWD}(a, n) = 1$, to

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Skoro pewna potęga daje resztę 1, to wykładnik wolno rozpatrywać modulo $\varphi(n)$ — wysoki wykładnik zwija się do małego i dalej liczy się zwykłym mnożeniem.

funkcja Eulera

Wartość $\varphi(n)$ to liczba tych spośród $1, \dots, n$, które są względnie pierwsze z n . Liczy się ją z rozkładu na czynniki pierwsze wzorem $\varphi(n) = n \prod_{p|n} (1 - 1/p)$, w którym iloczyn przebiega różne dzielniki pierwsze, niezależnie od tego, w jakiej potędze występują. Dla $n = 12 = 2^2 \cdot 3$ dostajemy $\varphi(12) = 12 \cdot \frac{1}{2} \cdot \frac{2}{3} = 4$. Warto pamiętać dwa przypadki szczególne: $\varphi(p) = p - 1$ oraz $\varphi(p^k) = p^k - p^{k-1}$.

przykład rachunkowy

Policzmy $7^{100} \pmod{12}$. Najpierw sprawdzamy założenie: $\text{NWD}(7, 12) = 1$, więc twierdzenie Eulera wolno zastosować. Mamy $\varphi(12) = 4$, a $100 = 4 \cdot 25$, zatem $7^{100} = (7^4)^{25} \equiv 1 \pmod{12}$. Gdyby wykładnik nie

dzielił się równo, liczymy resztę z dzielenia go przez $\varphi(n)$ i tylko tę resztę dopotęgujemy ręcznie.

▷ *Rzeczywisty rząd elementu bywa mniejszy niż $\varphi(n)$: tutaj już $7^2 = 49 \equiv 1 \pmod{12}$. Twierdzenie Eulera daje wykładnik, który na pewno działa, niekoniecznie najmniejszy.*

! Bez względnej pierwszości teza jest po prostu nieprawdziwa i to jest najczęstsza pułapka w tym temacie. Weźmy $a = 2$, $n = 12$: mamy $\text{NWD}(2, 12) = 2 \neq 1$, a $2^{\varphi(12)} = 2^4 = 16 \equiv 4 \pmod{12}$, czyli nie 1. Gorzej — żadna potęga dwójki nigdy nie da reszty 1 modulo 12, bo wszystkie są parzyste.

odwrotność modularna

Odwrotnością a modulo n nazywamy takie b , że $ab \equiv 1 \pmod{n}$; jest ona odpowiednikiem dzielenia w arytmetyce reszt. Istnieje wtedy i tylko wtedy, gdy $\text{NWD}(a, n) = 1$, i nie przypadkiem jest to ten sam warunek co wyżej: skoro $a^{\varphi(n)} \equiv 1$, to $a^{\varphi(n)-1}$ jest właśnie szukaną odwrotnością.

4.11 Chińskie twierdzenie o resztach

co mówi

Jeśli moduły $n_1, \dots, n_r$ są parami względnie pierwsze, to układ kongruencji

$$x \equiv a_1 \pmod{n_1}, \quad \dots, \quad x \equiv a_r \pmod{n_r}$$

ma dokładnie jedno rozwiązanie modulo iloczyn $N = n_1 \cdots n_r$. Dwa słowa są tu najważniejsze: *dokładnie jedno*, bo rozwiązanie istnieje zawsze i jest jedyne w zakresie od 0 do $N - 1$, oraz *parami*, bo względna pierwszość musi zachodzić dla każdej pary modułów z osobna.

Prosty przykład: $x \equiv 2 \pmod{3}$ i $x \equiv 3 \pmod{5}$ dają $x \equiv 8 \pmod{15}$, co sprawdza się w pamięci. Praktyczna wartość twierdzenia polega na tym, że pozwala rozbić rachunek modulo dużą liczbę na kilka niezależnych rachunków modulo małe czynniki — reszty modulo N zachowują się dokładnie tak, jak krotki reszt modulo poszczególne n_i .

! Bez parami względnej pierwszości teza psuje się na dwa sposoby. Układ $x \equiv 1 \pmod{2}$, $x \equiv 2 \pmod{4}$ nie ma rozwiązania w ogóle, bo pierwsza kongruencja żąda liczby nieparzystej, a druga parzystej. Z kolei $x \equiv 1 \pmod{2}$, $x \equiv 3 \pmod{4}$ rozwiązania ma, ale modulo iloczyn 8 są dwa, a nie jedno.

rozszerzony algorytm Euklidesa

Zwykły algorytm Euklidesa liczy $\text{NWD}(a, n)$ przez kolejne dzielenia z resztą. Wersja rozszerzona odtwarza przy okazji współczynniki tożsamości Bezouta, czyli takie całkowite s i t , że

$$as + nt = \text{NWD}(a, n),$$

co robi się, cofając się po wykonanych dzieleniach.

Najważniejsze zastosowanie jest natychmiastowe. Jeśli $\text{NWD}(a, n) = 1$, to $as + nt = 1$, a czytając tę równość modulo n dostajemy $as \equiv 1$, czyli s jest odwrotnością a modulo n . To jest praktyczny sposób znajdowania odwrotności — szybszy niż potęgowanie do $\varphi(n)-1$ i działający także wtedy, gdy rozkładu n na czynniki nie znamy. Widać stąd również, dlaczego odwrotność istnieje wyłącznie przy $\text{NWD}(a, n) = 1$: gdy największy wspólny dzielnik jest większy, lewa strona tożsamości Bezouta jest zawsze jego wielokrotnością i jedyński nigdy nie da.

4.12 Działanie grupy na zbiorze

orbita i stabilizator

Grupa G działa na zbiorze X , gdy każdy jej element przestawia elementy X w sposób zgodny ze składaniem. Każdemu punktowi x przypisujemy wtedy dwa obiekty. **Orbita** to zbiór wszystkich punktów, do których da się x przenieść jakimś elementem grupy — odpowiedź na pytanie „dokąd można trafić”. **Stabilizator** to zbiór elementów grupy zostawiających x na miejscu — „ile ruchów nic z nim nie robi”. Obie liczby wiąże wzór

$$|G| = |\text{orbita}(x)| \cdot |\text{stabilizator}(x)|.$$

Intuicja: żeby przenieść x w konkretne inne miejsce, można użyć dokładnie tylu elementów grupy, ile liczy stabilizator, więc grupa rozpada się na tyle równych paczek, ile jest punktów w orbicie. Wystarczy zatem znać dwie z trzech liczb, a rozmiar orbity zwykle widać gołym okiem.

sześcian, czyli jedyny potrzebny przykład

Grupa obrotów sześcianu ma **24 elementy**. Najłatwiej policzyć to samym wzorem: obrót może przenieść wybraną ścianę na dowolną z sześciu, a po ustaleniu, gdzie trafiła, zostają jeszcze cztery obroty wokół osi przez jej środek. Z odbiciami lustrzanymi grupa wszystkich symetrii ma **48 elementów**.

Dla wierzchołka rozumowanie wygląda tak: obroty przestawiają wszystkie osiem wierzchołków między sobą, więc orbita ma 8 elementów, a stabilizator liczy $24/8 = 3$ — faktycznie są to obroty wokół długiej przekątnej przez ten wierzchołek, o 120° , 240° i o zero stopni. Dla ściany orbita ma 6 elementów, więc stabilizator liczy $24/6 = 4$: obroty wokół osi przebiegającej jej środek. Dla krawędzi orbita ma 12 elementów, więc stabilizator liczy $24/12 = 2$, czyli tożsamość oraz obrót o 180° wokół osi przez środki dwóch przeciwległych krawędzi.

sześcian w liczbach: Grupa obrotów ma 24 elementy, z odbiciami 48. Wierzchołki: orbita 8, stabilizator 3. Ściany: orbita 6, stabilizator 4. Krawędzie: orbita 12, stabilizator 2. Każda para mnoży się do 24.

! Pułapki polegają tu prawie zawsze na podstawieniu liczby dotyczącej innego obiektu: stwierdzenie mówi o stabilizatorze wierzchołka, a podaje 4 (wartość dla ściany), albo o orbicie krawędzi, a podaje 8. Odruch obronny jest jeden — przemnoż podane liczby i sprawdź, czy wychodzi 24.

lemat Burnside’a

Lemat Burnside’a odpowiada na pytanie, ile jest *istotnie różnych* obiektów, gdy uznajemy za identyczne te, które przechodzą jeden w drugi przy działaniu grupy. Mówi, że liczba orbit jest *średnią* liczbą punktów stałych: dla każdego elementu grupy liczymy, ile obiektów zostawia on nieruszonych, sumujemy i dzielimy przez rząd grupy. Uśrednianie koryguje to, że obiekt o wielu symetriach byłby policzony wielokrotnie. Klasyczny wynik: kolorowań ścian sześcianu dwoma kolorami, z dokładnością do obrotu, jest dokładnie 10, podczas gdy bez uwzględniania obrotów byłoby $2^6 = 64$.

4.13 Grafy: fakty, które wracają

cykl Eulera i cykl Hamiltona

Cykl Eulera przechodzi przez każdą *krawędź* dokładnie raz i ma bardzo wygodne kryterium: istnieje wtedy i tylko wtedy, gdy graf jest spójny (pomijając wierzchołki izolowane) i *wszystkie* stopnie są parzyste. Intuicja jest natychmiastowa — skoro do wierzchołka trzeba wejść i z niego wyjść, krawędzie muszą łączyć się w pary. Wersja otwarta, czyli ścieżka bez powrotu do startu, dopuszcza dokładnie dwa wierzchołki nieparzystego stopnia; są to jej końce.

Cykl Hamiltona przechodzi przez każdy *wierzchołek* dokładnie raz i jest zupełnie inną historią. Proste kryterium dla niego nie ma — problem jest NP-zupełny. Istnieją tylko warunki *wystarczające*: twierdzenie Diraca (graf o $n \geq 3$ wierzchołkach, w którym każdy stopień wynosi co najmniej $n/2$, ma cykl Hamiltona) oraz twierdzenie Ore’a (wystarczy, by dla każdej pary niesąsiednich wierzchołków suma stopni wynosiła co najmniej n).

! To rozróżnienie jest najważniejsze w całym temacie: warunki Diraca i Ore’a działają tylko w jedną stronę. Graf, który ich nie spełnia, może mieć cykl Hamiltona i zwykle go ma — najprostszy przykład to sam cykl o dziesięciu wierzchołkach, gdzie każdy stopień wynosi 2, czyli daleko poniżej $n/2 = 5$. Zdanie „graf ma cykl Hamiltona wtedy i tylko wtedy, gdy spełnia warunek Diraca” jest fałszywe.

dwudzielnosc

Graf jest dwudzielny wtedy i tylko wtedy, gdy *nie zawiera cyklu nieparzystej długości*. To jest pełna charakteryzacja, działająca w obie strony, więc na teście bardzo wygodna: żeby obalić dwudzielnosc, wystarczy wskazać jeden trójkąt albo jeden pięciokąt. Za intuicją stoi kolorowanie dwoma kolorami — idąc wzdłuż cyklu, kolory zmieniają się na przemian, więc po powrocie do startu długość musi być parzysta.

planarnosc

Dla spójnego grafu planarnego narysowanego bez przecięć zachodzi wzór Eulera $n - m + f = 2$, gdzie n to liczba wierzchołków, m krawędzi, a f ścian, przy czym ścianę zewnętrzną też się liczy. Praktycznie ważniejszy jest wniosek: dla $n \geq 3$ każdy graf planarny spełnia $m \leq 3n - 6$, a jeśli dodatkowo nie ma trójkątów, to nawet $m \leq 2n - 4$. Jest to szybki test *nieplanarnosci* — graf o zbyt wielu krawędziach planarny być nie może.

Tak właśnie obala się dwa najsłynniejsze przykłady. Graf pełny K_5 ma $n = 5$ i $m = 10$, podczas gdy $3n - 6 = 9$, więc planarny nie jest. Graf pełny dwudzielny $K_{3,3}$ ma $n = 6$ i $m = 9$; trójkątów nie zawiera, bo jest dwudzielny, więc obowiązuje go ostrzejszy limit $2n - 4 = 8$, a krawędzi ma dziewięć.

! Nierówność $m \leq 3n - 6$ jest warunkiem *koniecznym*, nie *wystarczającym*. Jej spełnienie niczego nie dowodzi — istnieją grafy nieplanarne o niewielkiej liczbie krawędzi, choćby $K_{3,3}$ z dodatkowymi wierzchołkami wstawionymi na krawędziach.

twierdzenie Halla

Ostatni fakt dotyczy skojarzeń w grafie dwudzielnym o częściach A i B . Skojarzenie nasycające całe A , czyli przypisanie każdemu elementowi A osobnego partnera z B , istnieje wtedy i tylko wtedy, gdy dla *każdego* podzbioru $S \subseteq A$ zbiór jego sąsiadów jest co najmniej tak liczny jak samo S .

Obrazowo: jeśli mamy grupę osób i listę prac, które każda z nich potrafi wykonać, to da się rozdzielić prace bez konfliktów dokładnie wtedy, gdy żadna grupa k osób nie tłoczy się wokół mniej niż k prac.

Istotą warunku jest to, że musi zachodzić dla wszystkich podzbiorów naraz — jeden ciasny podzbiór psuje całe skojarzenie, choćby reszta grafu wyglądała bardzo bogato.

- ▷ Zdanie „każdy wierzchołek z A ma co najmniej jednego sąsiada, więc skojarzenie nasycające A istnieje” jest fałszywe — warunek dla pojedynczych wierzchołków to za mało.

4.14 Kolorowanie grafów

liczba chromatyczna

Pokolorować graf to przypisać wierzchołkom kolory tak, żeby sąsiedzi nigdy nie mieli tego samego. Najmniejsza liczba kolorów, która wystarcza, to **liczba chromatyczna** $\chi(G)$. Dwa przypadki brzegowe: graf bez krawędzi ma $\chi = 1$, a graf pełny K_n potrzebuje $\chi = n$, bo każde dwa wierzchołki sąsiadują.

Z poprzedniej sekcji wynika natychmiast, że graf mający choć jedną krawędź jest dwudzielny wtedy i tylko wtedy, gdy $\chi(G) = 2$ — podział na dwie części to przecież dokładnie pokolorowanie dwoma kolorami. Kryterium „brak cykli nieparzystej długości” jest więc zarazem kryterium dwukolorowości.

trzy twierdzenia, które wracają

Twierdzenie o czterech barwach mówi, że każdy graf planarny da się pokolorować czterema kolorami. Dotyczy *tylko* grafów planarnych — dla grafów ogólnych żadna stała nie wystarcza, bo K_n wymaga n kolorów.

Twierdzenie Brooksa ogranicza liczbę chromatyczną przez największy stopień Δ : dla spójnego grafu, który nie jest grafem pełnym ani cyklem nieparzystej długości, zachodzi $\chi(G) \leq \Delta$. Wyjątki nie są ozdobnikiem, tylko sednem, bo w obu tych przypadkach faktycznie potrzeba $\Delta + 1$ kolorów — stwierdzenie „dla każdego grafu $\chi \leq \Delta$ ” jest zatem fałszywe.

Twierdzenie Vizinga dotyczy kolorowania *krawędzi*, gdzie różne kolory muszą dostać krawędzie mające wspólny koniec. Mówi rzecz zaskakująco mocną: indeks chromatyczny $\chi'(G)$ przyjmuje jedną z zaledwie dwóch wartości, Δ albo $\Delta + 1$. Nic pośredniego ani większego się nie zdarza.

twierdzenie Königa

W grafach dwudzielnych wiele rzeczy robi się łatwiejszych i twierdzenie Königa jest tego dobrym przykładem: w grafie dwudzielnym rozmiar największego skojarzenia równa się rozmiarowi najmniejszego pokrycia wierzchołkowego, czyli najmniejszego zbioru wierzchołków dotykającego wszystkich krawędzi. Jest to dwudzielny odpowiednik twierdzenia Halla — Hall mówi, *kiedy* skojarzenie nasycające istnieje, a König podaje *rozmiar* największego skojarzenia, i to wyrażony przez wielkość zupełnie innego obiektu.

ROZDZIAŁ 5

Analiza matematyczna

analiza matematyczna: Analiza kosztuje kilkadziesiąt razy więcej nauki niż Python, a warta jest tyle samo punktów — więc jedyne sensowne podejście to ostra selekcja. Autorzy testu prawie nigdy nie każą doprowadzić rachunku do końca; zamiast tego podstawiają zdanie o *brzegu* przedziału zbieżności, o tym, czy jakaś wartość funkcji jest „niemożliwa”, albo zaczynające się od „dla każdej funkcji”. Wszystkie trzy typy rozstrzyga się w pamięci, jeśli wiadomo, gdzie leży pułapka — i te miejsca zebrałem w tym rozdziale.

5.1 Zbieżność ciągów bez rachunku

kryterium ilorazowe

Najskuteczniejszym narzędziem do rozstrzygania granic ciągów w pamięci jest patrzenie na *iloraz kolejnych wyrazów*, czyli na a_{n+1}/a_n : dzielenie wyrazu przez poprzedni skraca silnie do jednego czynnika, potęgi o stałej podstawie do samej podstawy, a wielomiany do wyrażeń dążących do jedynki.

Reguła jest taka: jeśli $a_n > 0$ i iloraz dąży do granicy q , to przy $q < 1$ ciąg dąży do zera, a przy $q > 1$ do nieskończoności. Bierze się to stąd, że gdy każdy wyraz jest najwyżej q razy większy od poprzedniego, cały ogon ciągu leży pod ciągiem geometrycznym o ilorazie q , a ten zbiega do zera.

cztery ciągi, które wracają najczęściej

Pierwszy to $n!/n^n$. Iloraz upraszcza się do $(n/(n+1))^n = 1/(1+1/n)^n$, a to dąży do $1/e \approx 0,37$, czyli poniżej jedynki — ciąg dąży do zera. Mniej formalnie: n^n to iloczyn n czynników równych n , a $n!$ iloczyn n czynników w większości znacznie mniejszych.

Drugi to $2^n/n^2$. Iloraz to $2 \cdot (n/(n+1))^2$, co dąży do 2, a więc ciąg dąży do nieskończoności. To kanoniczny zapis faktu, że wykładnicza bije potęgową, niezależnie od wykładnika w mianowniku.

Trzeci to $n!/2^n$, gdzie iloraz jest wręcz banalny: wynosi $(n+1)/2$, czyli rośnie nieograniczenie, więc ciąg dąży do nieskończoności. Silnia mnoży za każdym krokiem przez coraz większą liczbę, a wykładnicza wciąż przez tę samą dwójkę — i w pewnym momencie ucieka.

Czwarty to $n^2/n!$, czyli sytuacja odwrotna. Iloraz to $(n+1)/n^2$, co dąży do zera, więc ciąg dąży do zera — i widać przy okazji, że silnia bije nie tylko kwadrat, ale każdą ustaloną potęgę.

Ciąg	Iloraz a_{n+1}/a_n	Granica ciągu
$n!/n^n$	$\rightarrow 1/e \approx 0,37$	0
$2^n/n^2$	$\rightarrow 2$	$+\infty$
$n!/2^n$	$(n+1)/2 \rightarrow \infty$	$+\infty$
$n^2/n!$	$(n+1)/n^2 \rightarrow 0$	0

hierarchia rządów wzrostu

Z tych czterech rachunków składa się jedna reguła, która obsługuje nie tylko ciągi, ale też szeregi i złożoność algorytmów:

$$\ln n \ll n^a \ll c^n \ll n! \ll n^n \quad (a > 0, c > 1).$$

Czytamy to tak: logarytm rośnie wolniej niż jakakolwiek potęga, potęga wolniej niż dowolna wykładnicza, wykładnicza wolniej niż silnia, a silnia wolniej niż n^n . Symbol $\ll$ znaczy tu „iloraz dąży do zera”. Gdy w zadaniu pojawia się iloraz dwóch takich wyrażeń, odpowiedź czyta się z listy bez rachunku: wygrywa to, co stoi bardziej w prawo. Ostatnie ogniwo dopisuję osobno, bo o nim zapomina się najczęściej – silnia bywa mylnie traktowana jako najszybsza rzecz, jaka istnieje.

! Kryterium milczy przy granicy ilorazu równej 1: ciągi n , $1/n$ i 1 mają iloraz dążący do jedynki, a granice kolejno ∞ , 0 i 1.

5.2 Szeregi z parametrem i końce przedziału zbieżności

To najbardziej „pułapkogenny” temat w całej analizie, więc rozłożmy go bardzo dokładnie. Chodzi o szeregi postaci

$$\sum_{n=1}^{\infty} \frac{x^n}{n^k},$$

gdzie k jest ustalonym parametrem, a pytanie brzmi: dla jakich x szereg jest zbieżny. Ten schemat wraca regularnie, bo z jednego wzoru da się ułożyć cztery niezależne stwierdzenia, i to takie, w których prawdziwa odpowiedź zmienia się przy najmniejszej zmianie k .

promień zbieżności zawsze wynosi jeden

Zacznijmy od tego, co od k nie zależy. Zastosujmy kryterium d’Alemberta do wartości bezwzględnych: iloraz kolejnych wyrazów to $|x| \cdot (n/(n+1))^k$, a druga część dąży do jedynki dla każdego ustalonego k . Zostaje samo $|x|$. Wniosek: dla $|x| < 1$ szereg jest zbieżny, i to bezwzględnie, a dla $|x| > 1$ rozbieżny – *niezależnie od k* . Parametr k zmienia tylko czynnik wielomianowy, a wielomian jest za słaby, żeby wpłynąć na granicę ilorazu. Dla $|x| > 1$ widać to nawet bez kryterium: z hierarchii rządów wzrostu wiemy, że x^n bije n^k , więc wyrazy rosną i nie spełniają nawet warunku koniecznego.

prawy koniec przedziału

Podstawmy $x = 1$. Dostajemy $\sum 1/n^k$, czyli **szereg Dirichleta**, i tu odpowiedź jest ostra: jest zbieżny dokładnie wtedy, gdy $k > 1$. Dla $k = 1$ mamy szereg harmoniczny, rozbieżny, ale tak leniwie, że intuicja tu zawodzi – i właśnie dlatego $k = 1$ jest ulubionym podpunktem autorów. Dla $k = \frac{1}{2}$ wyrazy są jeszcze większe, więc tym bardziej rozbieżny, a dla $k = 2$ szereg jest zbieżny.

lewy koniec przedziału

Teraz $x = -1$. Dostajemy $\sum (-1)^n/n^k$, czyli szereg naprzemienny, a do takich mamy kryterium Leibniza: wystarczy, że wartości bezwzględne wyrazów maleją monotonicznie do zera. Ciąg $1/n^k$ maleje do zera dla *każdego* $k > 0$, więc na lewym końcu szereg jest zbieżny już przy $k > 0$. To ogromna różnica: dla $k = \frac{1}{2}$ w $x = 1$ szereg był rozbieżny, a tu jest zbieżny.

Trzeba tylko wiedzieć, jakiego rodzaju jest ta zbieżność. Dla $0 < k \leq 1$ jest ona *tylko warunkowa*, bo szereg wartości bezwzględnych to znów rozbieżny $\sum 1/n^k$ – zbieżność bierze się wyłącznie ze znoszenia się wyrazów dodatnich i ujemnych, a nie z tego, że są małe. Dla $k \leq 0$ wartości bezwzględne wyrazów nie dążą do zera, więc odpada nawet warunek konieczny i lewy koniec też jest rozbieżny.

trzy możliwe zbiory zbieżności

Wnętrze przedziału jest ustalone raz na zawsze, a końce dokładają się według reguł powyżej, więc zbiór zbieżności może być tylko jednym z trzech przedziałów:

Parametr	Zbiór zbieżności	Co się dzieje na końcach
$k \leq 0$	$(-1, 1)$	oba końce odpadają
$0 < k \leq 1$	$[-1, 1)$	lewy warunkowo, prawy rozbieżny
$k > 1$	$[-1, 1]$	oba bezwzględnie

asymetria końców przedziału: Lewy koniec jest „łatwiejszy” od prawego, bo zbieżność naprzemienną dostaje się taniej: wystarczy $k > 0$, a nie $k > 1$. Dlatego przedział zbieżności takiego szeregu *nigdy* nie jest otwarty z lewej i domknięty z prawej. Każde stwierdzenie mówiące, że zbiorem zbieżności jest $(-1, 1]$, jest fałszywe bez żadnego rachunku.

5.3 Kryteria zbieżności szeregów w praktyce

trzy, których używa się naprawdę

Warunek konieczny mówi, że wyrazy szeregu zbieżnego dążą do zera, i pracuje wyłącznie w drugą stronę: gdy wyrazy do zera nie dążą, szereg jest rozbieżny – ale samo dążenie do zera niczego nie gwarantuje, czego dowodem jest rozbieżny szereg harmoniczny $\sum 1/n$, i to jest najczęściej wykorzystywana pułapka w tym dziale. Kryterium d’Alemberta liczy granicę q ilorazu $|a_{n+1}/a_n|$ i przy $q < 1$ orzeka zbieżność bezwzględną, przy $q > 1$ rozbieżność, a przy $q = 1$ milczy; rozpoznaje się je po silniach i potęgach o stałej podstawie, bo te skracają się w ilorazie. Kryterium Leibniza wymaga, by w szeregu naprzemiennym wartości bezwzględne wyrazów malały monotonicznie do zera, i oba te założenia są potrzebne.

pozostałe, po pół zdania

Kryterium porównawcze – gdy wyraz da się oszacować przez szereg Dirichleta albo geometryczny. Kryterium Cauchy’ego, z pierwiastkiem $\sqrt[n]{|a_n|}$ – gdy cały wyraz jest n -tą potęgą. Kryterium całkowe, porównujące $\sum f(n)$ z $\int f$ – gdy f jest dodatnia i malejąca; to z niego bierze się warunek $k > 1$ dla Dirichleta. Rozróżnienie na zbieżność bezwzględną i warunkową ilustruje szereg anharmoniczny $1 - \frac{1}{2} + \frac{1}{3} - \dots$, który sumuje się do $\ln 2$, choć bez znaków jest rozbieżny.

5.4 Twierdzenie Lagrange’a jako maszyna do nierówności

co ono mówi

Jeśli f jest ciągła na $[a, b]$ i różniczkowalna wewnątrz, to istnieje punkt c między a i b taki, że

$$f'(c) = \frac{f(b) - f(a)}{b - a}.$$

Po prawej stronie stoi średnie tempo zmiany na całym przedziale, po lewej – tempo chwilowe w jednym punkcie. Geometrycznie: gdzieś na przedziale styczna jest równoległa do siecznej łączącej końce wykresu. Po ludzku: jeśli przejechałeś trasę ze średnią prędkością 80 km/h, to w pewnej chwili prędkościomierz wskazywał dokładnie tyle.

jak się z niego wyciska nierówności

Cała wartość egzaminacyjna tego twierdzenia siedzi w tym, że pozwala przełożyć wiedzę o *pochoďnej* na wiedzę o *wartościach*. Przypuśćmy, że wiemy tylko tyle, że $m \leq f'(x) \leq M$ na całym przedziale. Twierdzenie mówi, że przyrost funkcji równa się $f'(c)(b - a)$ dla pewnego c , a o tym $f'(c)$ wiemy, że mieści się między m i M . Stąd natychmiast

$$m(b - a) \leq f(b) - f(a) \leq M(b - a).$$

Zobaczmy to na liczbach. Niech $f(0) = 0$ i niech $2 \leq f'(x) \leq 3$ dla wszystkich x . Dla przedziału $[0, 5]$ powyższa nierówność daje $2 \cdot 5 \leq f(5) - 0 \leq 3 \cdot 5$, czyli $f(5)$ musi leżeć w przedziale $[10, 15]$; wartość 16 jest więc niemożliwa. Podobnie dla $[0, 4]$ wychodzi $f(4) \in [8, 12]$, więc $f(4)$ nie może przekroczyć 12.

„niemożliwe” a „może się zdarzyć”

Tu jest właściwa pułapka, bo autorzy testu grają na niej częściej niż na czymkolwiek innym w rachunku różniczkowym. Założenia dają nam *domknięty* przedział wartości dopuszczalnych. O liczbie z zewnątrz możemy powiedzieć, że jest niemożliwa; o liczbie z wewnątrz – wyłącznie z samymi końcami – nie możemy powiedzieć nic poza tym, że jest dopuszczalna.

Skąd wiadomo, że końce naprawdę są osiągalne? Bo realizuje je funkcja liniowa: dla $f(x) = 3x$ mamy $f' \equiv 3$, co spełnia założenie, i wychodzi $f(5) = 15$ oraz $f(4) = 12$. Skrajne wartości nie są więc przypadkiem granicznym do odrzucenia – są osiągalne.

jak czytać stwierdzenie o wartości brzegowej: Zdanie „z założeń wynika, że $f(4) \leq 12$ ” jest *prawdziwe*. Zdanie „wartość $f(4) = 12$ jest niemożliwa” jest *fałszywe*. Oba mówią o tej samej dwunastce i różnią się tylko tym, po której stronie granicy ją stawiają. Reguła praktyczna: policz dopuszczalny przedział i sprawdzaj wyłącznie, czy podana liczba do niego należy. Jeśli należy, każde zdanie o jej niemożliwości jest fałszem, choćby leżała na samym brzegu.

5.5 Funkcje wypukłe

definicja przez cięciwę i wniosek o całce

Funkcja jest wypukła, gdy dla dowolnych dwóch punktów wykresu odcinek je łączący leży nad wykresem, czyli $f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y)$ dla $t \in [0, 1]$; dla funkcji dwukrotnie różniczkowalnej jest to równoważne warunkowi $f'' \geq 0$.

Stąd wniosek, który wraca w zadaniach. Dla f wypukłej na $[0, 1]$ z warunkami $f(0) = 0$ i $f(1) = 1$ cięciwa jest prostą $y = x$, więc $f(x) \leq x$ na całym przedziale, a po scałkowaniu $\int_0^1 f \leq \frac{1}{2}$.

Z dołu natomiast nie ma żadnego ograniczenia i odruchowe uzupełnienie „czyli całka leży między 0 i $\frac{1}{2}$ ” jest fałszywe. Funkcja wypukła z tymi warunkami może w środku zanurkować dowolnie głęboko: łamana, która z $(0, 0)$ opada stromo do $(\frac{1}{2}, -M)$ i wraca do $(1, 1)$, ma rosnące nachylenia, więc jest wypukła, a jej całka wynosi około $-M/2$.

5.6 Wzór Taylora i rozwinięcia w zerze

Taylor przybliża funkcję wielomianem uzgodnionym z nią co do wartości i kolejnych pochodnych w punkcie. Potrzebna jest z tego przede wszystkim lista rozwinięć w zerze.

sześć rozwinięć w zerze:

$$\begin{aligned} e^x &= 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \dots, & \frac{1}{1-x} &= 1 + x + x^2 + x^3 + \dots \\ \sin x &= x - \frac{x^3}{6} + \frac{x^5}{120} - \dots, & \cos x &= 1 - \frac{x^2}{2} + \frac{x^4}{24} - \dots \\ \ln(1+x) &= x - \frac{x^2}{2} + \frac{x^3}{3} - \dots, & (1+x)^a &= 1 + ax + \frac{a(a-1)}{2}x^2 + \dots \end{aligned}$$

Szereg dla $\ln(1+x)$ ma w mianownikach n , a nie $n!$, i to jest najczęstsza pomyłka. Użycie: w granicy $\lim_{x \rightarrow 0} (x - \sin x)/x^3$ licznik redukuje się do $\frac{x^3}{6} + \dots$, więc $\frac{1}{6}$ widać od razu, a de l'Hospitalem trzeba by różniczkować trzykrotnie.

5.7 Kanoniczne kontrprzykłady

Duża część stwierdzeń w teście zaczyna się od „dla każdej funkcji”. Takie zdanie obala się jednym przykładem, więc nie trzeba nic dowodzić — wystarczy mieć w zapasie odpowiedni okaz. Poniżej cztery, które załatwiają większość przypadków.

ciągła, ale nie jednostajnie ciągła

Zwykła ciągłość jest własnością lokalną: dla każdego punktu i każdego ε istnieje δ , które może zależeć od punktu. Jednostajna ciągłość wymaga więcej — jedno δ musi wystarczyć dla wszystkich punktów naraz, a to psuje się tam, gdzie funkcja coraz szybciej drga albo się zadziera. Funkcja $\sin(1/x)$ na $(0, 1)$ jest ciągła, ale blisko zera oscyluje coraz szybciej, więc żadne ustalone δ nie utrzyma wahań w ryzach; x^2 na prostej też nie jest jednostajnie ciągła, bo daleko od zera jest tak stroma, że wartość skacze dowolnie mocno. Oba przykłady musiały mieć przedział otwarty albo nieograniczony: na domkniętym i ograniczonym ciągłość jest automatycznie jednostajna.

różniczkowalna o nieciągłej pochodnej

Funkcja $f(x) = x^2 \sin(1/x)$ dla $x \neq 0$ i $f(0) = 0$ jest różniczkowalna w każdym punkcie, także w zerze, gdzie $f'(0) = 0$, bo iloraz różnicowy jest ograniczony przez $|x|$. Ale dla $x \neq 0$ pochodna wynosi $2x \sin(1/x) - \cos(1/x)$, a drugi składnik przy $x \rightarrow 0$ skacze między -1 i 1 bez ustalania się na cokolwiek. Pochodna istnieje więc wszędzie i nie jest ciągła w zerze, co obala zdania typu „różniczkowalność daje ciągłą pochodną”.

zbieżność punktowa bez jednostajnej

Ciąg funkcyjny $f_n(x) = x^n$ na $[0, 1]$ zbiega punktowo do funkcji równej zero dla $x < 1$ i jedynie dla $x = 1$. Każda f_n jest ciągła, a granica — nie. Zbieżność nie jest jednostajna, bo dla każdego n można podejść tak blisko jedynki, że x^n jest bliskie jedności, więc różnica w kresie sięga wciąż całej jedynki. Tak obala się zdania „granica punktowa funkcji ciągłych jest ciągła” oraz „granicę można zawsze przenieść pod znak całki lub pochodnej”.

pochodne cząstkowe bez ciągłości

Funkcja $f(x, y) = xy/(x^2 + y^2)$ z wartością 0 w punkcie $(0, 0)$ znika tożsamościowo na obu osiach, więc obie pochodne cząstkowe w zerze istnieją i są równe zero. Mimo to funkcja nie jest w zerze ciągła: idąc do zera po prostej $y = x$ dostajemy stałe $x^2/(2x^2) = \frac{1}{2}$. Istnienie pochodnych cząstkowych nie daje więc ani ciągłości, ani różniczkowalności, bo mówi tylko o dwóch kierunkach; potrzebna jest ich *ciągłość*.

5.8 Funkcje wielu zmiennych

gradient, punkty krytyczne i hesjan

Gradient to wektor pochodnych cząstkowych i wskazuje kierunek najszybszego wzrostu funkcji, a jego długość mówi, jak szybki ten wzrost jest; na tym opiera się spadek gradientowy, który idzie w kierunku $-\nabla f$. W ekstremum leżącym wewnątrz obszaru gradient musi się zerować, ale to warunek wyłącznie konieczny — punkt siodłowy też go spełnia. Rozstrzyga dopiero macierz drugich pochodnych cząstkowych: dodatnio określona daje minimum lokalne, ujemnie określona maksimum, a nieokreślona punkt siodłowy — określoność bada się tak jak dla form kwadratowych w algebrze.

więzy i współrzędne biegunowe

Przy ekstremum na krzywej $g = 0$ poziomica f musi być styczna do więzu, bo inaczej dałoby się przesunąć wzdłuż niego i poprawić wartość. Styczność oznacza równoległość gradientów, skąd mnożniki Lagrange’a: rozwiązujemy $\nabla f = \lambda \nabla g$ wraz z $g = 0$. Podstawienie biegunowe $x = r \cos \varphi$, $y = r \sin \varphi$ daje zaś $dx dy = r dr d\varphi$, bo prostokącik dr na $d\varphi$ odpowiada wycinkowi pierścienia o boku kątowym długości $r d\varphi$ — im dalej od środka, tym dłuższy łuk przy tym samym przyroście kąta.

ROZDZIAŁ 6

Algorytmy i struktury danych

algorytmy: Ten dział jest na teście prawda/fałsz wyjątkowo dochodowy, bo prawie każde stwierdzenie da się rozstrzygnąć bez liczenia — wystarczy wiedzieć, na czym opiera się poprawność algorytmu i co się psuje, gdy któreś założenie odpadnie. Autorzy testu wracają przy tym do kilku stałych chwytów: podmieniają Θ na $\mathcal{O}$ albo odwrotnie, podstawiają wzór z niewłaściwej gałęzi równania rekurencyjnego, każą uwierzyć, że najmniejsze drzewo AVL rośnie jak 2^h , albo dokleją do prawdziwego zdania o czasie działania drugi, fałszywy członek o efekcie działania. To jeden z dwóch najważniejszych rozdziałów tych notatek, więc materiału jest tu dużo — ale ułożonego tak, żeby każdy temat dał się przeczytać osobno. I zaczynamy nie od tematu, a od umiejętności: od czytania pseudokodu, którego nigdy wcześniej nie widziałeś.

6.1 Jak czytać nieznany pseudokod

W arkuszu pojawiło się zadanie z algorytmem, którego nie ma w żadnym podręczniku — po prostu ktoś go wymyślił na tę okazję. Nie da się takiego zadania zrobić z pamięci, ale da się je zrobić rzemiosłem, i to zawsze tym samym. Poniższe cztery kroki wykonuj w tej kolejności.

1. **Zobacz, jak zmienia się zmienna licznikowa.** To jedyna rzecz, która decyduje o liczbie obrotów pętli. Jeżeli licznik rośnie o 1, pętla robi około n obrotów. Jeżeli jest *mnożony* — przez 2, przez 3, przez cokolwiek — obrotów będzie tylko około $\log n$, bo licznik startujący z 1 i podwajany dobija do n po $\log_2 n$ krokach. Ta jedna obserwacja odpowiada za większość różnicy między czasem n^2 a $n \log n$.
2. **Zobacz, które indeksy tablicy są w ogóle dotykane.** To krok, który najczęściej rozstrzyga zadanie, i prawie nikt go nie robi. Jeżeli pętla odwiedza tylko indeksy 1, 2, 4, 8, ..., to przy $n = 100$ dotknie ośmiu komórek na sto, a pozostałe dziewięćdziesiąt dwie zostaną dokładnie tam, gdzie były. Algorytm, który nie zajrzał do komórki, nie mógł jej przestawić, więc *nie ma żadnej możliwości*, żeby posortował całą tablicę — i to wiesz, nie prześledziwszy ani jednego porównania.
3. **Rozpoznaj, co robi jeden przebieg pętli wewnętrznej.** Wewnątrz prawie zawsze siedzi coś znanego, tylko bez nazwy. Porównanie sąsiadów $A[i]$ z $A[i+1]$ i zamiana, gdy stoją w złej kolejności, to pojedynczy przebieg sortowania bąbelkowego. Szukanie najmniejszego elementu w ogonie tablicy i wstawianie go na wolne miejsce z przodu to przebieg sortowania przez wybieranie. Nazwij to, a od razu wiesz, co jeden przebieg osiąga — i nie musisz już śledzić go krok po kroku.
4. **Policz, ile takich przebiegów trzeba, żeby efekt był pełny.** Tu zamyka się rachunek. Jeden przebieg bąbelkowy przesuwaa na koniec największy element z nieposortowanej części, czyli załatwia *jedną* pozycję. Żeby uporządkować całą tablicę, trzeba więc $n - 1$ przebiegów. Jeżeli pętla zewnętrzna wykonuje ich mniej, tablica nie musi wyjść posortowana — i zwykle nie wychodzi.

przykład do końca

Zobaczmy to na czymś konkretnym. Poniższy algorytm wygląda niewinnie:

```
DZIWNE_SORTOWANIE(A, n):
    j = 1
    while j < n:
        for i = 0 to n-2:
            if A[i] > A[i+1]:
                swap A[i], A[i+1]
        j = 2 * j
```

Krok pierwszy: licznik j jest mnożony przez 2, więc pętla zewnętrzna kręci się około $\log_2 n$ razy, a nie n razy. Krok drugi: pętla wewnętrzna przechodzi po *wszystkich* parach sąsiadów, więc tutaj żadna komórka nie zostaje pominięta — ten trop nic nie daje, wada jest gdzie indziej. Krok trzeci: wewnątrz to podręcznikowy przebieg sortowania bąbelkowego, koszt $n - 1$ porównań. Razem czas działania to $\Theta(n \log n)$ — i to jest prawda.

Krok czwarty rozstrzyga sprawę. Przebiegów bąbelkowych jest $\log_2 n$, a do pełnego posortowania potrzeba ich $n - 1$. Dla $n = 8$ pętla wykona przebiegi dla $j = 1, 2, 4$, czyli trzy, a potrzeba siedmiu. Weźmy tablicę odwróconą 8, 7, 6, 5, 4, 3, 2, 1: po pierwszym przebiegu jest 7, 6, 5, 4, 3, 2, 1, 8, po drugim 6, 5, 4, 3, 2, 1, 7, 8, po trzecim 5, 4, 3, 2, 1, 6, 7, 8 — i algorytm kończy pracę z tablicą, która oczywiście nie jest posortowana. Wniosek: czas $\Theta(n \log n)$ tak, posortowanie nie.

! Stwierdzenia w takich zadaniach są prawie zawsze *koniunkcjami*: „algorytm działa w czasie $\Theta(n \log n)$ i sortuje tablicę”. Koniunkcja jest fałszywa, gdy fałszywy jest choć jeden człon, więc nie musisz sprawdzać obu — wystarczy obalić jeden. I warto zacząć od drugiego, bo autorzy budują takie zdania tak, że część o czasie jest policzona uczciwie, a wywrotka siedzi w części o efekcie.

6.2 Duże O jako obietnica, która łatwo wybacza

asymetria górnej granicy

Zapis $\mathcal{O}(f)$ jest *ograniczeniem górnym* i niczym więcej. Zdanie „algorytm działa w czasie $\mathcal{O}(f)$ ” znaczy dokładnie tyle: nie gorzej niż f , z dokładnością do stałej. Nie mówi, że tak właśnie długo trwa; nie mówi, że oszacowanie jest ciasne. Dlatego zdanie „sortowanie przez scalanie działa w czasie $\mathcal{O}(n^2)$ ” jest *prawdziwe* — mimo że faktyczny czas to $\Theta(n \log n)$, a $n \log n$ rośnie wolniej od n^2 , więc obietnica „nie gorzej niż n^2 ” jest spełniona z zapasem. Odruch mówi „fałsz, bo to zły rząd”, i właśnie na tym odruchu zarabiają autorzy testu.

Notacja Θ takich rzeczy nie wybacza, bo znaczy „ani lepiej, ani gorzej”: jest jednocześnie ograniczeniem górnym i dolnym. Zdanie „sortowanie przez scalanie działa w czasie $\Theta(n^2)$ ” jest już fałszywe. Symetrycznie Ω jest samym ograniczeniem dolnym i wybacza w drugą stronę — $\Omega(n)$ dla sortowania jest prawdą, tylko bezużyteczną.

jak z tego korzystać przy rozstrzyganiu: Stwierdzenie z $\mathcal{O}$ obalisz *tylko* pokazując, że algorytm jest w istocie *wolniejszy* niż podana granica. Samo „ale on jest szybszy” niczego nie obala — przeciwnie, potwierdza zdanie. Stwierdzenie z Θ obalisz z obu stron: wystarczy, że prawdziwy rząd jest inny.

- ▷ Jest jeszcze druga oś tego samego rozróżnienia: o którym przypadku mowa. Zdanie „sortowanie szybkie działa w czasie $\mathcal{O}(n \log n)$ ” jest prawdą o przypadku średnim i fałszem o najgorszym, a to dwa różne stwierdzenia. Gdy zdanie nie precyzuje, przyjmuje się najgorszy przypadek — i tej umowy trzymaj się na teście.

6.3 Podstawowe struktury danych

stos i kolejka

Obie trzymają elementy i różnią się *tylko* tym, który wychodzi pierwszy: stos oddaje ostatnio wrzucony, kolejka tego, który czeka najdłużej, a obie operacje kosztują czas stały. Stos jest naturalnym kształtem *rekursji*, bo wywołanie, które zaczęło inne, musi poczekać na jego koniec — maszyna trzyma ramki wywołań właśnie na stosie — oraz *nawrotów*, bo ze ślepego zaułka wraca się do ostatniej decyzji, czyli tej na wierzchu. Kolejka odpowiada *przeglądaniu wszerek*, bo obsługuje wierzchołki w kolejności odkrycia, więc wszystko w odległości jednej krawędzi wypada przed czymkolwiek w odległości dwóch.

kolejka priorytetowa i kopiec binarny

Kolejka priorytetowa to *abstrakcja*: wstaw element, podejrzyj najmniejszy, usuń najmniejszy. Kopiec binarny jest jej najczęstszą realizacją — drzewo binarne z warunkiem na relację *rodzic-dziecko*, w wersji minimum takim, że rodzic jest nie większy od obu swoich dzieci. Warunek dotyczy wyłącznie pionu, więc kopiec *nie* jest drzewem poszukiwań i przegląd in-order nie daje z niego ciągu posortowanego; gwarantuje tylko minimum w korzeniu. Jako drzewo zupełne trzyma się w tablicy bez wskaźników i ma wysokość $\lfloor \log_2 n \rfloor$, więc podejrzenie minimum jest darmowe, a wstawianie i usunięcie minimum kosztują $\mathcal{O}(\log n)$.

budowanie kopca zajmuje czas liniowy

Z gotowej tablicy kopiec powstaje w czasie $\Theta(n)$, a nie $\Theta(n \log n)$, co zaskakuje, bo n wstawień po $\log n$ dawałoby to drugie. Przesiewamy w dół wierzchołki od ostatniego rodzica wstecz do korzenia, a rachunek wychodzi liniowo, bo przesianie wierzchołka z wysokości k kosztuje $\mathcal{O}(k)$, ale takich wierzchołków jest tylko około $n/2^{k+1}$. Prawie cały kopiec siedzi *nisko*, gdzie droga w dół jest krótka — połowa wierzchołków to liście, które nie kosztują nic — więc suma $\sum_k (n/2^{k+1}) \cdot \mathcal{O}(k)$ wychodzi $\mathcal{O}(n)$.

- ! Ta sama struktura ma więc *dwa* koszty budowy: n osobnych wstawień to $\Theta(n \log n)$, a przesiewanie od dołu $\Theta(n)$, dlatego zdanie „zbudowanie kopca wymaga czasu $n \log n$ ” jest fałszywe. Konsekwencją jest sortowanie przez kopcowanie: kopiec w $\Theta(n)$, potem n zdjęć minimum po $\mathcal{O}(\log n)$, czyli $\Theta(n \log n)$ *gwarantowane* i bez dodatkowej pamięci, choć bez stabilności.

6.4 Drzewa poszukiwań binarnych i ich uogólnienia

warunek BST i przegląd in-order

W każdym wierzchołku wszystkie klucze w lewym poddrzewie są mniejsze od jego klucza, a wszystkie w prawym większe. Mowa o *całych poddrzewach*, a nie tylko o dwóch dzieciach — i na tym skrócie autorzy testu budują fałszywe stwierdzenia. Stąd też przegląd in-order — lewe poddrzewo, wierzchołek, prawe — daje ciąg posortowany: wypisuje najpierw wszystko mniejsze, potem sam klucz, na końcu wszystko większe, a to samo rozumowanie stosuje się rekurencyjnie w poddrzewach.

degeneracja do listy

Warunek BST nie mówi nic o kształcie, a wysokość zależy od kolejności wstawiania. Wstaw klucze $1, 2, 3, \dots, n$ dokładnie w tej kolejności: każdy kolejny jest większy od wszystkiego, co już jest, więc idzie w prawo i drzewo robi się *ścieżką* – zwykłą listą o wysokości $n - 1$, w której wyszukiwanie kosztuje $\Theta(n)$.

! Słynne $\mathcal{O}(\log n)$ dla drzewa poszukiwań nie jest więc bezwarunkowe: obowiązuje dla drzew *zrównoważonych* albo dla danych losowych. Stwierdzenie „wyszukiwanie w BST działa w czasie logarytmicznym” bez słowa o równoważeniu jest fałszywe – a dane wstawiane po kolei to przypadek najzupełniej naturalny, nie złośliwy wyjątek.

drzewa czerwono-czarne

Druga, obok AVL, odpowiedź na degenerację. Zrównoważenie jest tu *luźniejsze*: nie żąda się, by wysokości poddrzew różniły się o jeden, a tylko by najdłuższa ścieżka od korzenia do liścia nie przekraczała dwukrotności najkrótszej. Skutek jest wymienny – drzewo czerwono-czarne bywa *wyższe* od AVL o tej samej liczbie węzłów, więc wyszukiwanie jest nieco wolniejsze, ale wstawianie i usuwanie wymagają mniej obrotów, bo słabszy warunek psuje się rzadziej. Oba dają $\mathcal{O}(\log n)$.

B-drzewa i po co są

B-drzewo nie jest binarne: jeden wierzchołek trzyma wiele kluczy i ma wiele dzieci, więc przy stopniu rozgałęzienia B wysokość jest rzędu $\log_B n$. Sens widać, gdy dane nie mieszczą się w pamięci: odczyt bloku z dysku jest o kilka rzędów wielkości droższy od porównania, więc liczy się nie liczba porównań, a liczba *odczytów*. Wierzchołek dobiera się rozmiarem do jednego bloku dyskowego, więc zejście o poziom to jeden odczyt – przy stopniu rzędu setek drzewo z milionami kluczy ma wysokość trzy albo cztery. Dlatego na B-drzewach stoją bazy danych i systemy plików.

6.5 Drzewa AVL i ich wysokość

definicja

Drzewo AVL to drzewo poszukiwań binarnych z jednym dodatkowym warunkiem nałożonym w *każdym wierzchołku*: wysokości jego lewego i prawego poddrzewa różnią się o co najwyżej 1. To warunek lokalny i bardzo słaby – nie wymaga, żeby drzewo było pełne ani nawet w przybliżeniu symetryczne globalnie. A jednak wystarcza, żeby wysokość została logarytmiczna, i właśnie to jest ciekawe. Umawiamy się, że wysokość to liczba poziomów, więc jednowierzchołkowe drzewo ma wysokość 1.

największe drzewo danej wysokości

Tu nie ma nad czym myśleć: najwięcej wierzchołków przy wysokości h ma drzewo *pełne*, w którym każdy poziom jest zapełniony do końca. Poziomy mają $1, 2, 4, \dots, 2^{h-1}$ wierzchołków, co daje razem $2^h - 1$. Drzewo pełne spełnia oczywiście warunek AVL, bo wszystkie poddrzewa mają równe wysokości.

najmniejsze drzewo danej wysokości

To rozumowanie jest ważniejsze i mniej oczywiste. Chcemy zbudować drzewo AVL o wysokości h z jak najmniejszej liczby wierzchołków, czyli maksymalnie „rozchudzone”. Bieremy korzeń; jedno poddrzewo musi mieć wysokość $h - 1$, bo inaczej całe drzewo nie sięgnęłoby wysokości h . Drugie chcemy mieć jak najmniejsze, ale warunek AVL nie pozwala zejść niżej niż do $h - 2$. Oba poddrzewa robimy więc

najmniejszymi możliwymi, i dostajemy zależność

$$N(h) = 1 + N(h-1) + N(h-2), \quad N(0) = 0, \quad N(1) = 1,$$

czyli dokładnie rekurencję Fibonacciego z przesunięciem. Kolejne wartości to 1, 2, 4, 7, 12, 20, 33, ..., a domknięty wzór to $N(h) = F(h+2) - 1$.

to jest miejsce, w którym robi się podmiang: Najmniejsze drzewo AVL o wysokości h ma około φ^h wierzchołków, gdzie $\varphi \approx 1,618$ – a nie około 2^h . Liczba $2^h - 1$ to rozmiar drzewa *największego*, czyli pełnego. Typowe fałszywe stwierdzenie polega na wstawieniu jednej z tych liczb w miejsce drugiej.

stąd logarytmiczna wysokość

Wniosek wyciąga się przez odwrócenie nierówności. Drzewo AVL o n wierzchołkach i wysokości h spełnia

$$F(h+2) - 1 \leq n \leq 2^h - 1,$$

bo nie może być mniejsze od najmniejszego ani większe od największego. Lewa strona rośnie wykładniczo, z podstawą φ (dokładniej jak $\varphi^{h+2}/\sqrt{5}$), więc jeśli n jest co najmniej tak duże, to h nie może przekroczyć $\log_\varphi n$ z małą poprawką, czyli około $1,44 \log_2 n$. Innymi słowy: wysokość może być gorsza niż w drzewie idealnie zrównoważonym, ale najwyżej o stały czynnik – a stały czynnik nie zmienia rzędu, więc wyszukiwanie, wstawianie i usuwanie kosztują $\mathcal{O}(\log n)$.

- ▷ *Liczbowo widać, jaki jest zapas: drzewo AVL o wysokości 6 ma co najmniej 20 wierzchołków i co najwyżej 63. To bardzo szeroka rozpiętość – i właśnie w niej mieszka cała elastyczność tej struktury.*

6.6 Tablice haszujące i słowniki

idea haszowania i kolizje

Zamiast szukać, *liczysz*, gdzie rzecz leży: funkcja haszująca przerabia klucz na indeks w tablicy, więc żeby sprawdzić, czy klucz jest w słowniku, nie porównujesz go z niczym – wyliczasz adres i zaglądasz w jedną komórkę. Możliwych kluczy jest jednak więcej niż komórek, więc z zasady szufladkowej dwa różne klucze muszą kiedyś trafić w to samo miejsce. Na taką kolizję są dwie strategie: *łańcuchowanie*, gdzie w komórce trzymasz listę wszystkich kluczy, które tam wpadły, i przeglądasz ją liniowo, oraz *adresowanie otwarte*, gdzie wszystko siedzi w samej tablicy, a przy zajętej komórce szukasz następnej wolnej według ustalonej reguły – na przykład kolejnej w prawo.

koszt oczekiwany kontra najgorszy

Przy dobrej funkcji haszującej i pilnowanym zapełnieniu klucze rozkładają się równomiernie, łańcuchy są krótkie i koszt operacji jest *oczekiwany* stały. Gwarancji jednak nie ma: gdy wszystkie klucze zahaszują się w to samo miejsce, łańcuchowanie degeneruje się do jednej długiej listy i wyszukiwanie kosztuje $\Theta(n)$. Dlatego zdanie „tablica haszująca wyszukuje w czasie stałym” jest prawdziwe jako zdanie o koszcie oczekiwanym i fałszywe jako zdanie o najgorszym przypadku – ta sama asymetria co przy sortowaniu szybkim.

hasz kontra drzewo zrównoważone

To zestawienie jest w tym temacie najcenniejsze, bo właśnie z niego robi się stwierdzenia testowe. Hasz jest szybszy, koszt stały wobec logarytmicznego, ale płaci za to utratą *porządku*, bo z założenia rozrzuca podobne klucze w odległe miejsca. Nie odpowie więc na żadne pytanie porządkowe: nie wypiszesz z niego kluczy z zakresu od 10 do 50, nie znajdziesz następnika danego klucza ani elementu najmniejszego, a przejście po zawartości daje kolejność przypadkową. Drzewo zrównoważone jest wolniejsze o czynnik logarytmiczny, ale każde z tych pytań obsługuje, bo trzyma dane w porządku.

6.7 Sortowania: tylko to, co rozstrzyga stwierdzenia

Nie ma sensu pamiętać tabelki ośmiu algorytmów, bo stwierdzenia na teście kręcą się wokół czterech spraw.

dolne ograniczenie i jego założenie

Każdy algorytm, który o kolejności elementów dowiaduje się *wyłącznie* z porównań, potrzebuje w najgorszym razie $\Omega(n \log n)$ porównań. Argument jest ładny: przebieg takiego algorytmu to schodzenie po drzewie decyzyjnym, w którym każdy wierzchołek to jedno porównanie z dwoma wyjściami. Skoro algorytm musi rozróżnić wszystkie $n!$ możliwych ustawień danych, drzewo musi mieć co najmniej $n!$ liści, a drzewo binarne o tylu liściach ma wysokość przynajmniej

$$\log_2(n!) = \Theta(n \log n).$$

Wysokość drzewa to liczba porównań w najgorszym przypadku, i stąd teza.

dlaczego zliczanie i pozycyjne nie łamią tej granicy

Bo w ogóle nie stają do tego wyścigu: nie porównują elementów, tylko korzystają z ich *wartości*. Sortowanie przez zliczanie używa wartości jako indeksu w tablicy liczników, sortowanie pozycyjne rozdziela elementy po kolejnych cyfrach. Żadne z nich nie schodzi po drzewie decyzyjnym z porównań, więc dowód ich nie obejmuje — dlatego mogą działać w czasie liniowym, ale tylko przy założeniu o danych, na przykład że to liczby całkowite z niezbyt szerokiego zakresu.

! Stwierdzenie „żaden algorytm nie sortuje szybciej niż w czasie $n \log n$ ” jest fałszywe, bo brakuje w nim słowa „oparty na porównaniach”. Odwrotnie: „sortowanie pozycyjne sortuje dowolne dane w czasie liniowym” też jest fałszywe, bo pomija założenie o postaci danych. Obie wersje tego zdania krążą po testach.

stabilność

Sortowanie jest stabilne, gdy elementy o równych kluczach zostają po sortowaniu w tej samej kolejności, w jakiej były przed nim. Brzmi jak drobiazg, a ma konkretne zastosowanie: pozwala sortować po wielu kluczach przez wielokrotne sortowanie po jednym. Jeżeli posortujesz listę osób po imieniu, a potem *stabilnie* po nazwisku, to wewnątrz każdego nazwiska imiona zostaną uporządkowane — bo stabilność nie zniszczyła poprzedniej pracy. Przy sortowaniu niestabilnym ta sztuczka się rozsypuje; na tej własności opiera się zresztą poprawność sortowania pozycyjnego.

sortowanie szybkie i jego najgorszy przypadek

Quicksort ma świetny czas średni $\Theta(n \log n)$ i bardzo małe stałe, ale w najgorszym przypadku działa w czasie $\Theta(n^2)$ — gdy podział wypada skrajnie nierówno, na przykład przy naiwnym wyborze piwotu na tablicy już posortowanej. To najczęstsze źródło fałszywych stwierdzeń w tym temacie: zdanie „sortowanie szybkie działa w czasie $\mathcal{O}(n \log n)$ ” bez dopisku „średnio” czytamy jako zdanie o najgorszym przypadku, a więc jako fałsz. Dla kontrastu sortowanie przez scalanie ma $\Theta(n \log n)$ *gwarantowane*, zawsze, bo dzieli tablicę na pół niezależnie od danych — płaci za to dodatkową pamięcią rzędu n na scalanie.

6.8 Selekcja, czyli k -ty element bez sortowania

Quickselect

Żeby znaleźć k -ty najmniejszy element, wystarczyłoby posortować i odczytać pozycję k — ale to praca ponad potrzebę, bo sortowanie ustala kolejność *wszystkich* par. Quickselect wybiera piwot i dzieli tablicę jak sortowanie szybkie, tylko po podziale wie, w której części leży pozycja k , i schodzi rekurencyjnie *tylko tam*, porzucając drugą połowę w całości. Zamiast dwóch podproblemów rozwiązuje jeden, więc koszty sumują się do $n + n/2 + n/4 + \dots < 2n$ i oczekiwany czas jest *liniowy*. Ceną jest to samo co u Quicksorta: przy pechowych piwotach wychodzi $\Theta(n^2)$.

mediana median

Wariant z gwarancją: dzieli się dane na piątki, liczy medianę każdej piątki, a potem rekurencyjnie medianę tych median. Taki piwot zawsze odcina stałą część tablicy, więc czas jest liniowy *w najgorszym przypadku* — praktycznie bywa jednak wolniejszy od Quickselecta, bo stała ukryta w tym $\mathcal{O}$ jest duża.

6.9 Techniki algorytmiczne

metoda nawrotów

Systematyczne przeszukiwanie przestrzeni rozwiązań: budujesz rozwiązanie częściowe, a gdy tylko widać, że nie da się go domknąć, ucinasz gałąź i wracasz do ostatniej decyzji. Rekursja jest naturalna, bo „wróć do ostatniej decyzji” to dokładnie wyjście z wywołania. Cała oszczędność siedzi w *odcinaniu*: w zadaniu o n hetmanach stawiasz po jednym hetmanie w każdym wierszu i przed zejściem głębiej sprawdzasz, czy nowy nie jest atakowany — jeśli jest, nie schodzisz wcale, a to jednym ruchem wyrzuca całe poddrzewo ustawień. Odcinanie zmienia jednak tylko stałą, nie klasę złożoności: w najgorszym przypadku nawroty są wykładnicze.

dziel i zwyciężaj

Dzielisz zadanie na kilka mniejszych tego samego typu, rozwiązujesz je rekurencyjnie i scalasz wyniki. Koszt spełnia wtedy zależność $T(n) = aT(n/b) + f(n)$, czyli dokładnie tę postać, którą obsługuje twierdzenie o rekurencji uniwersalnej: porównujesz $f(n)$ z $n^{\log_b a}$ i wygrywa większe z nich. Różnica wobec programowania dynamicznego jest jedna: tutaj podproblemy są *rozłączne*, więc nikt niczego nie liczy dwa razy i tablica nie jest potrzebna.

algorytmy zachłanne

Bierzesz to, co wygląda najlepiej lokalnie, i nigdy nie wracasz do decyzji. Najważniejsza rzecz w tym temacie jest taka: zachłanność jest poprawna *tylko wtedy*, gdy problem ma odpowiednią strukturę. Nie jest metodą, która zawsze działa — jest hipotezą, którą trzeba dla danego problemu udowodnić, zwykle argumentem wymiany.

Najlepiej zapamiętać to na kontraście trzech zadań. Wybór największej liczby zajęć z danymi początkami i końcami: brać zawsze zajęcie kończące się *najwcześniej* — działa, bo zajęcie zwalniające salę najszybciej zostawia najwięcej miejsca dla pozostałych. Plecak ciągły, gdzie przedmioty wolno dzielić: brać w kolejności malejącego stosunku wartości do wagi — też działa, bo pojemność wypełnisz dokładnie, dopełniając ją ułamkiem ostatniego przedmiotu. Plecak 0-1, gdzie przedmiot bierze się cały albo wcale: ta sama reguła *nie* działa, bo dopełnić się nie da. Kontrprzykład jest mały: pojemność 10, przedmiot *A* o wadze 6 i wartości 12, oraz *B* i *C*, każdy o wadze 5 i wartości 9. Stosunki to 2 dla *A* i 1,8 dla pozostałych, więc zachłannie bierzemy *A*, po czym zostają 4 jednostki i nic się już nie mieści — wynik 12. Optimum to *B* i *C* razem: waga 10, wartość 18.

kod Huffmana

Klasyczny przykład zachłanności *dowodliwie* optymalnej. Budujesz drzewo kodu od dołu: bierzesz dwa najrzadsze symbole, scalasz je w jeden wierzchołek o częstości równej sumie, wrzucasz z powrotem do puli i powtarzasz, aż zostanie jeden. Symbole częste zostają blisko korzenia i dostają krótkie słowa kodowe, a wynik jest kodem bezprefiksowym o najmniejszej oczekiwanej długości.

6.10 Programowanie dynamiczne i najdłuższy wspólny podciąg

idea w jednym zdaniu

Zamiast rozwiązywać problem dla całych danych, rozwiązujesz go dla wszystkich *prefiksów*, zapisujesz wyniki w tablicy i każdy kolejny wynik składasz z sąsiadów, którzy już są policzeni.

W zadaniu o najdłuższym wspólnym podciągu $C[i, j]$ oznacza długość najdłuższego wspólnego podciągu pierwszych i znaków słowa X i pierwszych j znaków słowa Y . Równanie ma dwie gałęzie i to na nich stoi całe zadanie:

$$C[i, j] = \begin{cases} 1 + C[i - 1, j - 1], & x_i = y_j, \\ \max(C[i - 1, j], C[i, j - 1]), & x_i \neq y_j. \end{cases}$$

Sens jest prosty. Gdy ostatnie znaki obu prefiksów są takie same, wolno je sparować i dopisać do najlepszego rozwiązania dla tego, co zostało po odcięciu obu — stąd 1 i cofnięcie się o jeden w *obu* indeksach. Gdy się różnią, to co najmniej jeden z tych znaków nie należy do wspólnego podciągu, więc próbujesz odciąć raz jeden, raz drugi i bierzesz lepszy wynik.

! Pułapka jest tu zawsze ta sama i bardzo mechaniczna: autor podstawia wzór z niewłaściwej gałęzi.

Najczęstsze wersje to $\max$ powiększony o jeden w przypadku niezgodnych znaków oraz $1 + C[i - 1, j - 1]$ wpisane tam, gdzie znaki się różnią. Czytając stwierdzenie, sprawdź najpierw *warunek* przy gałęzi, a dopiero potem wzór.

dlaczego komórka kosztuje czas stały

Bo jej policzenie to jedno porównanie znaków i jedno wzięcie maksimum z dwóch liczb — niezależnie od tego, jak wielkie są i i j . Kluczowe jest to, że wartości w tablicy *już streszczają* całe prefiksy: nie musisz

do nich zaglądać ponownie. Dlatego pierwszy odruch „komórka dla dużych indeksów patrzy na dłuższe słowa, więc musi być droższa” jest błędny, a cały algorytm kosztuje $\Theta(mn)$: tyle jest komórek, po stałej pracy na każdą.

najkrótszy wspólny nadciąg

Ten sam rachunek daje darmo drugi wynik. Nadciąg to słowo, w którym oba wejściowe słowa siedzą jako podciągi, i najkrótszy taki ma długość

$$i + j - C[i, j].$$

Słowami: żeby zmieścić oba słowa w jednym, wspólną część wystarczy zapisać *raz*, a wszystko poza nią trzeba dopisać osobno. Gdybyś zapisał je po prostu jedno po drugim, wyszłoby $i + j$ znaków, przy czym wspólny podciąg zostałby zapisany dwukrotnie — odejmując $C[i, j]$, usuwasz tę nadwyżkę.

6.11 Programowanie dynamiczne w pozostałych zadaniach

odległość edycyjna

Najmniejsza liczba operacji przerabiających jedno słowo na drugie, przy trzech dozwolonych: wstawieniu znaku, usunięciu znaku i zamianie znaku na inny. Równanie jest lustrzanym odbiciem tego od wspólnego podciągu, bo zamiast maksimum bierze minimum:

$$D[i, j] = \begin{cases} D[i-1, j-1], & x_i = y_j, \\ 1 + \min(D[i-1, j], D[i, j-1], D[i-1, j-1]), & x_i \neq y_j. \end{cases}$$

Trzy składniki minimum to trzy operacje: sięgnięcie w górę odpowiada usunięciu, w lewo wstawieniu, a na skos zamianie. Gdy znaki się zgadzają, nie płacisz nic i schodzisz na skos, a koszt całości to $\Theta(mn)$.

plecak 0-1 i pseudowielomianowość

Tablicę indeksuje się numerem przedmiotu i pozostałą pojemnością, a dwie gałęzie odpowiadają decyzji „biorę” albo „nie biorę”: bierzemy lepszą z możliwości $W[i-1, c]$ oraz $v_i + W[i-1, c - w_i]$, przy czym drugą tylko wtedy, gdy przedmiot się mieści. Komórek jest $n \cdot C$ po stałej pracy na każdą, więc czas to $\Theta(nC)$, gdzie C to pojemność. Tu jest subtelność, którą autorzy testu lubią: to *nie* jest algorytm wielomianowy, choć tak wygląda. Rozmiar danych mierzy się liczbą bitów, a pojemność C zapisana dwójkowo zajmuje tylko $\log C$ bitów, więc czas $\Theta(nC)$ jest wykładniczy względem *długości zapisu* danych. Mówimy wtedy o złożoności pseudowielomianowej — wielomianowej względem wartości liczby, a nie jej zapisu — i dlatego ten algorytm nie przeczy temu, że plecak 0-1 jest problemem NP-trudnym.

najdłuższy rosnący podciąg

Warto znać oba warianty, bo różnią się rzędem. Kwadratowy jest bezpośredni: $L[i]$ to długość najdłuższego rosnącego podciągu kończącego się na pozycji i , a liczy się ją, przeglądając wszystkie wcześniejsze pozycje o mniejszej wartości i biorąc najlepszą plus jeden, stąd $\Theta(n^2)$. Wariant $\Theta(n \log n)$ trzyma tablicę, w której na pozycji ℓ stoi najmniejsza możliwa wartość końcowa rosnącego podciągu długości ℓ ; taka tablica jest z natury rosnąca, więc dla nowego elementu wystarczy wyszukiwaniem binarnym znaleźć miejsce, w które trafia, i je nadpisać.

łańcuch macierzy oraz dwa kierunki rachunku

Mnożenie macierzy jest łączne, więc wynik nie zależy od nawiasowania — ale koszt zależy, i to o rzędy wielkości. Dla macierzy 10×100 , 100×5 i 5×50 mnożenie od lewej kosztuje 5000, a potem 2500, razem 7500 mnożeń; od prawej 25 000, a potem 50 000, razem 75 000 — dziesięć razy więcej za ten sam wynik. Optymalne nawiasowanie znajduje programowanie dynamiczne po przedziałach, w czasie $\Theta(n^3)$. Warto jeszcze rozróżnić dwa kierunki rachunku: podejście *zstępujące z memoizacją* to rekursja zapamiętująca wyniki, która liczy tylko podproblemy naprawdę potrzebne, ale płaci narzutem wywołań i głębokością stosu, natomiast *wstępujące* wypełnia tablicę w kolejności gwarantującej gotowość sąsiadów, bez rekursji i często oszczędniej pamięciowo — za to liczy wszystkie komórki, także zbędne.

6.12 Przeglądy grafu, porządki i spójność

przegląd wszerz

Odwiedza wierzchołki w kolejności rosnącej odległości od źródła, posługując się kolejką, i kosztuje $\Theta(n + m)$. Najważniejsza własność: w grafie *nieważonym* daje najkrótsze ścieżki, czyli o najmniejszej liczbie krawędzi. Powód jest wprost z konstrukcji kolejki — wszystko w odległości d zostaje odwiedzone, zanim ruszy cokolwiek w odległości $d + 1$, więc pierwszy moment dotarcia do wierzchołka jest zarazem najkrótszym.

Załamuje się to natychmiast, gdy krawędzie dostają różne wagi, bo ścieżka o wielu tanich krawędziach bywa krótsza od jednej drogiej — zdanie „przegląd wszerz znajduje najkrótsze ścieżki” jest prawdziwe tylko dla grafów nieważonych albo o wszystkich wagach równych.

przegląd w głąb

Schodzi tak głęboko, jak może, a potem się wycofuje; realizuje się go stosem wywołań rekurencyjnych, koszt również $\Theta(n + m)$. Daje dwie rzeczy. Po pierwsze wykrywanie cykli: jeżeli w grafie skierowanym natrafisz na krawędź prowadzącą do wierzchołka wciąż otwartego, czyli takiego, którego przetwarzanie zaczęliśmy i nie skończyliśmy, to znaczy, że domknąłeś cykl. Po drugie strukturę czasów odwiedzin — przedziały od wejścia do wyjścia są dla dowolnych dwóch wierzchołków albo rozłączne, albo jeden zawarty w drugim, nigdy zaś zachodzące częściowo.

sortowanie topologiczne

Ustawienie wierzchołków w ciąg taki, że każda krawędź prowadzi w przód. Istnieje *dokładnie* dla skierowanych grafów acyklicznych: gdyby był cykl, każdy jego wierzchołek musiałby stać przed pozostałymi, co jest niemożliwe, a odwrotnie — w grafie acyklicznym zawsze istnieje wierzchołek bez krawędzi wchodzących, więc można go wypisać, usunąć i powtórzyć. Równoważnie wystarczy odwrócić kolejność czasów wyjścia z przeglądu w głąb. Koszt $\Theta(n + m)$, a porządek na ogół nie jest jedyny. Stąd dwa wnioski na test: zdanie „każdy graf skierowany ma porządek topologiczny” jest fałszywe, a próba takiego sortowania jest gotowym testem acykliczności.

silnie spójne składowe

W grafie skierowanym wierzchołki u i v leżą w tej samej silnie spójnej składowej, gdy da się przejść z u do v i z powrotem. Relacja ta dzieli wierzchołki na rozłączne klasy, a po ściśnięciu każdej składowej do jednego wierzchołka otrzymany graf jest zawsze acykliczny — cykl między składowymi oznaczałby, że da się między nimi krążyć w obie strony, czyli że są jedną składową. Wyznacza się je w czasie $\Theta(n + m)$ algorytmem Kosaraju, robiącym dwa przeglądy w głąb, drugi na grafie z odwróconymi krawędziami.

6.13 Drzewa rozpinające, najkrótsze ścieżki i wagi ujemne

Ten kontrast warto znać dokładnie, bo o wagi ujemne pyta się chętnie, a odpowiedź nie jest dla obu problemów taka sama.

minimalne drzewo rozpinające nie boi się wag ujemnych

Kruskal, Prim i Borůvka opierają się na jednej własności, zwanej własnością cięcia: jeśli podzielisz wierzchołki na dwie niepuste części, to najlżejsza krawędź przechodząca między nimi należy do pewnego minimalnego drzewa rozpinającego. Zwróć uwagę, czego ten fakt *wymaga*: tylko tego, żeby dało się powiedzieć, która krawędź jest lżejsza. Korzysta więc wyłącznie z *porządku* wag, a nie z ich znaków ani wartości. Wagi ujemne porządku nie psują — -7 jest po prostu mniejsze od -2 — więc wszystkie trzy algorytmy działają na nich bez żadnej zmiany.

drugie uzasadnienie, przez liczbę krawędzi

Jest jeszcze jeden argument, warto znać oba. Każde drzewo rozpinające grafu o n wierzchołkach ma *dokładnie* $n - 1$ krawędzi — co do jednej, bez wyjątków. Jeśli więc dodasz stałą c do wszystkich wag, to waga każdego drzewa rozpinającego wzrośnie o $(n - 1)c$:

$$w'(T) = w(T) + (n - 1)c.$$

Wszystkim po równo, więc ich uporządkowanie się nie zmienia i najlżejsze zostaje najlżejszym. Biorąc c dostatecznie duże, każdy graf z wagami ujemnymi zamienisz więc na graf z wagami dodatnimi o tym samym rozwiązaniu.

dlaczego z Dijkstrą jest inaczej

Zauważ, że powyższy argument o dodawaniu stałej *nie przenosi się* na najkrótsze ścieżki, i to jest sedno różnicy: różne ścieżki mają różną liczbę krawędzi, więc dodanie stałej karze ścieżki długie mocniej niż krótkie i potrafi zmienić, która jest najkrótsza.

Sam algorytm Dijkstry psuje się z osobnego, ale pokrewnego powodu. Jego konstrukcja polega na tym, że w każdym kroku bierze wierzchołek o najmniejszej znanej odległości i *zamyka* go — uznaje jego odległość za ostateczną i nigdy do niej nie wraca. Wolno tak robić tylko wtedy, gdy wydłużanie ścieżki nie może jej skrócić, czyli gdy wszystkie wagi są nieujemne. Jedna krawędź ujemna to założenie łamie: ścieżka odkryta później może poprawić wynik dla wierzchołka już zamkniętego, a algorytm nie ma mechanizmu, żeby to zauważyć.

Najmniejszy przykład ma trzy wierzchołki. Ze źródła s prowadzą krawędzie $s \rightarrow a$ o wadze 2 i $s \rightarrow b$ o wadze 5, a dodatkowo jest krawędź $b \rightarrow a$ o wadze -4 . Dijkstra zamyka a z odległością 2, bo $2 < 5$ i wygląda to na wynik pewny — a prawdziwa odległość do a wynosi $5 - 4 = 1$, przez b . Do grafów z wagami ujemnymi używa się więc algorytmu Bellmana-Forda, który nic nie zamyka: po prostu $n - 1$ razy przegląda wszystkie krawędzie i próbuje poprawiać odległości, co daje czas $\Theta(nm)$.

algorytm Borůvky i skąd logarytm

Ten algorytm działa fazami i to jego cała idea. Na początku każdy wierzchołek jest osobnym drzewem. W jednej fazie *każde* obecne drzewo wybiera swoją najlżejszą krawędź wychodzącą na zewnątrz, po czym wszystkie wybrane krawędzie dokładamy naraz i scalamy to, co się połączyło. Liczba faz jest logarytmiczna, bo każde drzewo łączy się w fazie z co najmniej jednym innym, więc drzew zostaje najwyżej połowa — a połowienie od n do jednego zajmuje $\log_2 n$ kroków. Jedna faza to przegląd krawędzi, czyli

$\mathcal{O}(m)$, i stąd całość w czasie $\mathcal{O}(m \log n)$.

6.14 Wszystkie pary, przepływy i skojarzenia

Bellman-Ford i ujemne cykle

Liczba rund w tym algorytmie nie jest przypadkowa: najkrótsza ścieżka ma najwyżej $n - 1$ krawędzi, a po k -tej rundzie policzone są poprawnie wszystkie ścieżki o co najwyżej k krawędziach. Stąd druga umiejętność — uruchom jedną rundę więcej, a jeśli cokolwiek jeszcze się poprawi, w grafie jest cykl o ujemnej sumie wag. Wykrycie go jest istotne, bo przy takim cyklu „najkrótsza ścieżka” przestaje mieć sens: krążąc po nim, można schodzić dowolnie nisko.

Floyd-Warshall

Liczy odległości między *wszystkimi* parami wierzchołków naraz, w czasie $\Theta(n^3)$ i pamięci $\Theta(n^2)$. Pozwalasz ścieżkom korzystać po kolei z coraz większego zbioru wierzchołków pośrednich i po dopuszczeniu wierzchołka k poprawiasz każdą parę wzorem $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$. Radzi sobie z wagami ujemnymi, byle nie było ujemnych cykli, a ujemna liczba na przekątnej właśnie taki cykl zdradza. Kolejność pętli jest klasyczną pułapką: wierzchołek pośredni k musi być pętlą *zewnętrzną*, bo inaczej algorytm przestaje być poprawny, choć wygląda identycznie.

maksymalny przepływ i minimalny przekrój

Każda krawędź ma przepustowość, a Ty chcesz przepchnąć ze źródła s do ujścia t jak najwięcej, nie przekraczając przepustowości i nie gubiąc ani nie tworząc niczego w wierzchołkach pośrednich. Przekrój to podział wierzchołków na dwie części, jedną z s i drugą z t , a jego przepustowość to suma przepustowości krawędzi biegnących z pierwszej do drugiej. Twierdzenie mówi, że największy przepływ równa się najmniejszemu przekrojowi.

Jedna nierówność jest łatwa: cokolwiek dopłynie do t , musi w pewnym momencie przekroczyć granicę między częściami, więc przepływ nie może przewyższyć tego, co granica jest w stanie przepuścić. Druga jest sednem. Weź przepływ maksymalny i spójrz na graf resztowy, w którym każda krawędź ma tyle, ile jej jeszcze zostało, a dodatkowo istnieją krawędzie wstecz o tyle, ile już płynie. Skoro przepływ jest maksymalny, nie ma w tym grafie żadnej ścieżki z s do t , więc zbiór S wierzchołków wciąż osiągalnych z s nie zawiera t i wyznacza przekrój. I teraz dwie obserwacje: każda krawędź z S na zewnątrz musi być *wypełniona do końca*, bo inaczej zostałby na niej zapas i drugi koniec też należałby do S , a każda krawędź wchodząca do S musi być *pusta*, bo inaczej jej krawędź wstecz powiększyłaby S . Przez ten konkretny przekrój płynie więc dokładnie tyle, ile wynosi jego przepustowość — a skoro żaden przepływ nie przewyższa żadnego przekroju, ten przepływ jest największy, a ten przekrój najmniejszy.

skojarzenia w grafach dwudzielnych

Skojarzenie to zbiór krawędzi, w którym żaden wierzchołek nie powtarza się dwa razy. Żeby znaleźć największe, dorzuca się źródło połączone ze wszystkimi wierzchołkami lewej strony i ujście połączone ze wszystkimi z prawej, wszystkim krawędziom nadaje przepustowość 1 i pyta o maksymalny przepływ. Przepustowość 1 pilnuje, by każdy wierzchołek został użyty najwyżej raz, więc przepływ o wartości k to dokładnie skojarzenie o k krawędziach.

co która metoda wymaga

Warunki wejściowe są tym, co autorzy testu podmieniają najchętniej, więc zbierzmy je na koniec. Przegląd wszerek i w głąb działają zawsze, w czasie $\Theta(n + m)$; sortowanie topologiczne wymaga grafu acyklicznego, Dijkstra wag nieujemnych, a Bellman-Ford i Floyd-Warshall tylko braku ujemnych cykli. Kruskal i Prim potrzebują grafu nieskierowanego i dają $\mathcal{O}(m \log n)$, a Edmonds-Karp liczy maksymalny przepływ w czasie $\mathcal{O}(nm^2)$.

6.15 Wyszukiwanie wzorca w tekstach

algorytm naiwny

Przystawiasz wzorzec do każdej możliwej pozycji tekstu i porównujesz znak po znaku, a przy pierwszej niezgodności przesuwasz się o jedną pozycję i zaczynasz od nowa. W najgorszym przypadku kosztuje to $\Theta(nm)$, a zły przypadek jest łatwy: tekst z samych liter a i wzorzec z samych a z jednym b na końcu — na każdej pozycji dopasujesz prawie cały wzorzec i przewrócisz się na ostatnim znaku.

Knuth-Morris-Pratt i funkcja prefiksowa

Algorytm naiwny po częściowym dopasowaniu wyrzuca całą zdobytą wiedzę i cofa się w tekście. KMP nie cofa się nigdy, a narzędziem jest funkcja prefiksowa, liczona dla samego wzorca: dla każdej jego pozycji podaje długość najdłuższego właściwego prefiksu wzorca, który jest jednocześnie sufiksem dopasowanego dotąd fragmentu.

Dlaczego to pozwala nie wracać? Gdy pomylisz się po dopasowaniu k znaków, te k znaków tekstu są identyczne z pierwszymi k znakami wzorca, bo dokładnie tak się dopasowały. Jedyne, co trzeba jeszcze wiedzieć, to jaki najdłuższy początek wzorca ma szansę pasować do końcówki tego, co już przeczytałeś — a to jest właśnie wartość funkcji prefiksowej. Przesuwasz więc sam wzorzec, a wskaźnik w tekście zostawiasz na miejscu; skoro idzie on tylko w przód, każdy znak tekstu oglądasz stałą liczbę razy, stąd $\Theta(n)$ na przebieg i $\Theta(m)$ na policzenie funkcji.

Rabin-Karp

Zamiast napisów porównuje ich skróty: liczysz hasz wzorca i hasz każdego okna tekstu tej samej długości, a równość haszów oznacza kandydata, którego trzeba jeszcze potwierdzić znak po znaku. Cały pomysł stoi na haszowaniu *kroczącym* — gdy okno przesuwa się o jedną pozycję, nowy hasz wylicza się ze starego w czasie stałym, bo wystarczy odjąć wkład znaku wychodzącego i dodać wkład wchodzącego. Oczekiwany czas to $\Theta(n+m)$, ale najgorszy przypadek jest kwadratowy: przy pechowych kolizjach każde okno wymaga sprawdzenia znak po znaku.

Boyer-Moore i Aho-Corasick

Boyer-Moore porównuje wzorzec od końca i wykorzystuje niezgodność do dużego skoku w przód: jeśli znak tekstu, na którym się przewróciło, nie występuje we wzorcu w ogóle, to żadne przystawienie zachodzące na ten znak nie ma sensu i można przeskoczyć o całą długość wzorca. Część znaków nie zostaje wtedy nawet obejrzana, więc przy długich wzorcach i dużym alfabecie koszt bywa w praktyce *mniejszy* niż liniowy — i dlatego to on jest zwykle najszybszy w prawdziwych zastosowaniach, choć w wersji podstawowej najgorszy przypadek pozostaje kwadratowy. Aho-Corasick jest wersją dla wielu wzorców naraz: buduje z nich automat, czyli drzewo prefiksów z odsyłaczami w stylu KMP, i jednym przejściem po tekście znajduje wszystkie wystąpienia wszystkich wzorców.

6.16 Poprawność i koszt zamortyzowany

niezmiennik pętli

Podstawowe narzędzie do dowodzenia, że pętla robi to, co miała. Formułujesz warunek o zmiennych pętli i sprawdzasz go w trzech momentach: że jest prawdziwy przed pierwszym obrotem, że każdy obrót go zachowuje — jeśli był prawdziwy na początku obrotu, jest i na końcu — a stąd przez indukcję, że jest prawdziwy po wyjściu z pętli. Po to się to wszystko robi: z niezmiennika i warunku wyjścia razem wynika teza. Na sortowaniu przez wstawianie widać to w jednym zdaniu — niezmiennikiem jest „po i obrotach pierwsze i elementów jest posortowanych między sobą”, a warunek wyjścia daje $i = n$, więc posortowana jest cała tablica.

własność stopu oraz poprawność częściowa i całkowita

Niezmiennik mówi, co jest prawdą *jeśli* pętla się skończy — a że się skończy, trzeba pokazać osobno. Standardowy sposób: wskazać wielkość całkowitą, która przy każdym obrocie maleje i jest ograniczona z dołu. W wyszukiwaniu binarnym jest to długość przeszukiwanego przedziału, bo co obrót spada co najmniej o połowę i nie zejdzie poniżej zera, a coś, co maleje i ma dno, nie może maleć bez końca. Stąd rozróżnienie, o które testy pytają wprost: poprawność częściowa znaczy „jeśli algorytm się zatrzyma, wynik będzie poprawny”, a całkowita to poprawność częściowa plus gwarancja zatrzymania. Sam niezmiennik daje więc tylko połowę rzeczy.

koszt zamortyzowany

To średni koszt operacji w *najgorszym możliwym ciągu* operacji — a nie średnia po losowych danych, bo to zupełnie inne pojęcie. Weź tablicę dynamiczną podwajaną przy przepełnieniu: dopisanie zwykle kosztuje stałą, ale gdy tablica jest pełna, trzeba przydzielić dwa razy większą i przepisać całą zawartość, czyli zapłacić $\Theta(n)$ za jedną operację. Pointa jest w tym, jak rzadko to wypada. Przepisania zdarzają się przy rozmiarach $1, 2, 4, 8, \dots, n$, więc ich łączny koszt to $1 + 2 + 4 + \dots + n$, a suma potęg dwójki jest mniejsza od dwukrotności ostatniego składnika, czyli od $2n$. Na przepisywanie w ciągu n dopisań idzie więc mniej niż $2n$, czyli poniżej dwóch jednostek na dopisanie — koszt zamortyzowany $\Theta(1)$. Istotne jest samo podwajanie: gdyby tablica rosła o stałą liczbę komórek, przepisania byłoby proporcjonalnie do n , a ich suma wyszłaby $\Theta(n^2)$.

! Zwróć uwagę, czego to *nie* mówi. „Każde dopisanie do tablicy dynamicznej kosztuje czas stały” jest fałszywe, bo pojedyncze dopisanie bywa liniowe; „koszt zamortyzowany dopisania jest stały” jest prawdziwe. Zamortyzowanie to obietnica o całym ciągu operacji, nie o pojedynczej.

ROZDZIAŁ 7

Python

Python: Ten dział jest wart **16 punktów** — tyle samo, ile cała analiza matematyczna — i jest najtańszy w nauce na całym egzaminie. Znasz ten język zawodowo, więc nie ma tu powtarzania podstaw: zostały wyłącznie miejsca, w których Python zachowuje się inaczej, niż podpowiada nawyk, bo dokładnie z nich autorzy testu robią stwierdzenia. Egzamin sprawdza w tym dziale jedną umiejętność — wykonanie kilku linii kodu w *głowie* i podanie dokładnego wyniku albo nazwy wyjątku.

7.1 Zmienna jest etykietą, nie pudełkiem

Zmienna w Pythonie nie jest pudełkiem na wartość, tylko *etykietą przyklejoną do obiektu*, a przypisanie kopiuje referencję, nie obiekt. Dla obiektów niemutowalnych nie ma to znaczenia, ale dla list, słowników i tablic każda zmiana zrobiona przez jedną nazwę jest natychmiast widoczna przez każdą inną nazwę tego samego obiektu. Cała ta sekcja jest jedną konsekwencją tego faktu i jest to najczęściej wracający motyw całego działu.

Najwygodniej myśleć o tym tak, że lista nie przechowuje wartości, tylko spis adresów. Dwa różne wpisy w spisie mogą prowadzić pod ten sam adres, a wtedy sformułowania „zmieniłem element listy” i „zmieniłem obiekt, na który wskazuje element listy” oznaczają dwie zupełnie różne rzeczy — i właśnie na tej różnicy opierają się poniższe pułapki.

lista referencji do jednego wiersza

Chcemy macierz $N \times N$ z zerami i piszemy `A = N * [[0] * N]`. Wygląda niewinnie, poprawnie się wypisuje, a jest zepsute. Najpierw `[0] * N` tworzy *jeden* obiekt: listę N zer. Potem mnożenie przez N nie buduje N takich list — powtarza N razy *referencję* do tej jednej, która już istnieje. Mnożenie listy nigdy nie kopiuje jej elementów.

```
N = 3
A = N * [[0] * N]
A[0][0] = 13
print(A)                # [[13,0,0], [13,0,0], [13,0,0]]
print(A[0] is A[1] is A[2]) # True --- to JEDEN wiersz
```

Przypisanie `A[0][0] = 13` czyta się tak: weź obiekt `A[0]`, czyli ten jedyny wiersz, i zmień w nim element zerowy. Ponieważ `A[1]` i `A[2]` to nazwy tego samego wiersza, trzynastka pojawia się we wszystkich trzech miejscach naraz — z zewnątrz wygląda to tak, jakby jedna instrukcja zmieniła całą pierwszą *kolumnę*, i tym opisem autorzy testu chętnie się posługują. Poprawnie buduje się tę macierz wyrażeniem `[[0] * N for _ in range(N)]`, bo wtedy `[0] * N` wykonuje się od nowa N razy.

co zmienia numpy

Po przepuszczeniu tej samej listy przez `np.array` problem znika — nie dlatego, że numpy coś naprawia, ale dlatego, że tablica nie jest listą wskaźników do wierszy. Jest jednym ciągłym blokiem liczb, w którym każdy element ma własne miejsce, a konstruktor przepisuje tam wartości, więc powstaje N^2 niezależnych komórek. Po `B = np.array(A)` ta sama instrukcja `B[0][0] = 13` zmienia już dokładnie jeden element.

Ta różnica ma czytelny ślad w pamięci. Lista N referencji do jednego wiersza trzyma N wskaźników plus jeden wiersz, więc zajmuje $\mathcal{O}(N)$ — mimo że udaje obiekt o N^2 elementach. Tablica $N \times N$ trzyma wszystkie komórki, więc zajmuje $\mathcal{O}(N^2)$: dla $N = 1000$ to kilkanaście kilobajtów kontra megabajty. Warto zauważyć, w którą stronę to działa: uczciwie zbudowana lista list, ta z wyrażenia listowego, zajmuje oczywiście też $\mathcal{O}(N^2)$, więc oszczędność pamięci nie jest tu żadną zaletą — jest miarą tego, ile wierszy w ogóle nie ma.

7.2 Kopia, widok i maski w numpy

wycinek listy, wycinek tablicy

Wycinek listy tworzy *nową* listę, więc zapis do niej nie rusza oryginału. W numpy ta sama składnia daje **widok** — obiekt bez własnych danych, patrzący w ten sam blok pamięci co oryginał — więc zapis do widoku jest zapisem do oryginału. Powód jest wydajnościowy: przy dużych tablicach kopiowanie na każdym wycinku byłoby marnotrawstwem, dlatego numpy kopiuje wyłącznie na żądanie, metodą `.copy()`. Praktyczna konsekwencja jest taka, że funkcja, która dostaje wycinek tablicy i coś w nim zapisuje, po cichu modyfikuje dane wołającego — przy liście ta sama funkcja byłaby całkowicie bezpieczna.

```
L = [1, 2, 3, 4]
S = L[1:3]; S[0] = 99
print(L)           # [1, 2, 3, 4] --- lista bez zmian

T = np.array([1, 2, 3, 4])
V = T[1:3]; V[0] = 99
print(T)           # [ 1 99  3  4] --- tablica ZMIENIONA
```

! Jeśli w stwierdzeniu jest wycinek i zapis, pierwsze pytanie brzmi: lista czy tablica? Ta sama linia kodu ma dwie różne odpowiedzi. Osobno warto pamiętać, że kopia listy przez `L[:]` jest *płytką*, więc przy liście lista zmiana w głąb przez kopię i tak dotknie oryginału — na to jest `copy.deepcopy`.

maski liczą się na aktualnym stanie

Maska logiczna, na przykład `B % 2 == 1`, nie jest opisem warunku do sprawdzania na bieżąco — jest tablicą wartości logicznych wyliczoną *w chwili wykonania tej linii*. Jeśli więc wykonasz dwa przypisania z maskami jedno po drugim, drugie policzy maskę na tablicy *już zmienionej* przez pierwsze i kolejność zaczyna decydować o wyniku.

```
B = np.array([1, 2, 3, 4, 5, 6])
B[B % 2 == 1] = 0      # nieparzyste -> 0, wiec sa teraz PARZYSTE
B[B % 2 == 0] = 7      # ta maska obejmuje juz WSZYSTKO
print(B)               # [7 7 7 7 7 7]
```

Po pierwszej linii tablica to `[0 2 0 4 0 6]`, czyli same liczby parzyste, bo zero też jest parzyste. Druga maska sprawdza parzystość na tym nowym stanie, więc wybiera całą tablicę i wszystkie sześć elementów

dostaje siódmkę. Te same przypisania w odwrotnej kolejności dałyby `[0 0 0 0 0 0]`, a obie maski policzone z góry, przed jakimkolwiek zapisem, dałyby `[0 7 0 7 0 7]` — czyli dopiero to trzecie wykonanie robi to, czego się na pierwszy rzut oka oczekuje. Trzy różne wyniki z dwóch tych samych przypisań to bardzo wygodny materiał na stwierdzenia i dlatego przy każdej masce warto zapytać, na jakim stanie tablicy została policzona.

indeksowanie krotką

Dla tablicy zapisy `A[0][0]` i `A[0,0]` dają ten sam wynik, tylko różnymi drogami: pierwszy pobiera wiersz jako osobny obiekt i indeksuje ten wiersz, drugi od razu podaje pełny adres jako krotkę indeksów. Dla *listy* `list` działa wyłącznie pierwszy zapis — `L[0,0]` rzuca `TypeError`, bo indeksem listy może być tylko liczba całkowita albo wycinek, a nie krotka. Lista nie ma pojęcia wymiaru; jest jednowymiarowym pojemnikiem, w którym akurat leżą inne listy, i to właśnie dlatego nie wie, co miałyby zrobić z parą indeksów. Zwróć przy tym uwagę, że dla tablicy pośredni obiekt `A[0]` jest widokiem, a nie kopią wiersza, więc zapis przez `A[0][0]` zmienia tablicę dokładnie tak samo jak `A[0,0]`.

- ▷ Dwie rzeczy tylko do przypomnienia: *broadcasting* dopasowuje kształty od końca i wymaga, by każda para wymiarów była równa albo zawierała jedynkę, a *axis=0* sumuje po wierszach, czyli daje wynik dla każdej kolumny osobno.

7.3 Wyrażenia listowe i koszt kontenerów

kolejność klauzul

Klauzule `for` czyta się od lewej do prawej dokładnie jak zagnieżdżone pętle: pierwsza jest zewnętrzna, druga wewnętrzna, a wyrażenie na początku jest tym, co znalazłoby się w środku najgłębszej pętli. Dlatego `{e for x in L for e in x}` spłaszcza listę list — zewnętrzna pętla idzie po wierszach, wewnętrzna po elementach wiersza — i dla `L = [[1,2], [2,3], [3,4]]` daje zbiór `{1, 2, 3, 4}`. Ta sama treść w nawiasach kwadratowych dałaby listę `[1, 2, 2, 3, 3, 4]`, bo powtórzenia znikają dopiero dzięki klamrom, nie dzięki samemu spłaszczaniu.

Po odwróceniu kolejności klauzul pętla zewnętrzna próbuje iterować po `x`, które zostanie zdefiniowane dopiero w pętli wewnętrznej, czyli *później*. Nazwa nie istnieje w chwili, w której jest potrzebna, i leci `NameError`. Istotny jest tu moment: definicja funkcji kompiluje się bez protestu, bo Python sprawdza wtedy tylko składnię, a nie to, czy nazwy będą istnieć — wyjątek przychodzi dopiero przy *wywołaniu*. Najszybszy sposób rozstrzygnięcia takiego stwierdzenia to przepisać wyrażenie na jawne pętle w kolejności zapisu: jeśli pierwsza pętla odwołuje się do zmiennej, którą wprowadza dopiero druga, kod jest martwy.

```
def zla(L):
    return {e for e in x for x in L} # definicja przechodzi bez błędu

zla([[1, 2]]) # NameError: name 'x' is not defined
```

- ! Stwierdzenie „ten kod nie skompiluje się” jest tu *falszywe*, mimo że kod jest bezsensowny. Przy okazji: wersja z nawiasami okrągłymi to generator, więc jest jednorazowa i po wyczerpaniu daje pustą sekwencję, a nie wyjątek.

koszt przynależności

Wyrażenie `x in L` dla listy przechodzi ją element po elemencie, więc kosztuje $\mathcal{O}(n)$, a dla zbioru i słownika jest praktycznie natychmiastowe, średnio $\mathcal{O}(1)$, bo te struktury liczą z elementu hasz i idą prosto do miejsca, w którym musiałby leżeć. Warto raz przeliczyć skalę: przy dziesięciu milionach elementów i tysiącu sprawdzeń lista daje 10^{10} porównań, czyli realnie długie sekundy albo minuty, a zbiór tysiąc operacji o stałym koszcie, czyli czas nieodczuwalny. Samo zbudowanie zbioru kosztuje $\mathcal{O}(n)$, więc `S = set(L)` zwraca się już przy kilku sprawdzeniach — to jedna z tych zmian, które dotyczą jednej linii, a skracają czas działania o rzędy wielkości. Ceną jest to, że zbiór nie pamięta kolejności i nie przyjmuje duplikatów, więc jeśli któraś z tych rzeczy jest potrzebna, lista zostaje, ale przynajmniej wiadomo, za co się płaci.

Do tego szczegół o konstruktorach: `set(range(M))` i `list(range(M))` działają, bo zbiór i lista biorą z obiektu iterowalnego *pojedyncze elementy*, natomiast `dict(range(M))` rzuca `TypeError`, bo słownik potrzebuje *par* klucz-wartość, a z liczby 0 pary zrobić nie można. Kluczem może być tylko obiekt haszowalny, więc krotka tak, lista nie. I jeszcze jedna rzecz, o którą łatwo się potknąć przy szybkim czytaniu: dla słownika `in` sprawdza *klucze*, a nie wartości, więc `1 in {'a': 1}` jest fałszem.

7.4 Wyjątki: `TypeError` kontra `ValueError`

To najważniejsza para w całym dziale i jedyna, w której kryterium nie jest oczywiste. `TypeError` znaczy: argument jest złego *typu*, więc operacja nie ma sensu niezależnie od tego, jaka jest jego wartość. `ValueError` znaczy: typ jest poprawny i funkcja takie argumenty przyjmuje, ale *ta konkretna wartość* jest niedopuszczalna.

Najlepiej widać to na jednej funkcji. `int("3.5")` rzuca `ValueError`, bo napis jest właściwym typem argumentu dla `int` — `int("12")` działa świetnie — tylko ta konkretna treść nie jest zapisem liczby całkowitej. Natomiast `int([3])` rzuca `TypeError`, bo lista nie jest typem, z którego `int` potrafi cokolwiek zrobić, i żadna inna lista tego nie zmieni. Tak samo `len(5)` to `TypeError` (liczba nie ma długości i mieć jej nie będzie), a `L.index(9)` dla listy bez dziewiątki to `ValueError`: typ dobry, tylko tej wartości w liście nie ma.

▷ *Test: gdyby podstawić inną wartość tego samego typu, czy kod by zadziałał? Jeśli tak — `ValueError`; jeśli żadna by nie pomogła — `TypeError`.*

Kryterium warto sprawdzić na przypadku, w którym intuicja podpowiada odwrotnie: niedopasowanie kształtów w *broadcastingu* to `ValueError`, a nie `TypeError`, bo z typami jest wszystko w porządku — oba argumenty są tablicami liczb, niezgodne są dopiero wymiary, czyli wartości. Tak samo użycie wieloelementowej tablicy w `if` daje `ValueError`, bo nie wiadomo, czy taka tablica ma być prawdą, czy fałszem.

Z pozostałych wystarczy wiedzieć, kiedy lecą: `KeyError` przy nieobecnym kluczu słownika (ale `get` zwraca wtedy `None` i nie rzuca nic), `AttributeError` przy metodzie, której obiekt nie ma, `ZeroDivisionError` przy dzieleniu i reszcie przez zero (w *numpy* to już tylko ostrzeżenie i wynik `inf` albo `nan`), a `StopIteration` z `next` na wyczerpanym iteratorze — przy czym pętla `for` przechwytyuje go po cichu i kończy się normalnie.

7.5 Trzy drobiazgi, które zaskakują

arytmetyka

Dzielenie całkowite zaokrągla *w dół*, a nie w stronę zera, i znak reszty bierze się od dzielnika, więc `(-3) % 2` daje 1, a `-7 // 2` daje `-4`. Osobno: `bool` jest podklasą `int`, więc `True + True` daje 2, a `sum` po liście

wartości logicznych zlicza prawdy.

krótkie spięcie

or nie liczy drugiego członu, gdy pierwszy jest prawdziwy, a and nie liczy, gdy pierwszy jest fałszywy. Dlatego `len(L) >= 5 or L[99] == 9` nie rzuci `IndexError`, o ile lista ma co najmniej pięć elementów — gdyby była krótsza, pierwszy człon byłby fałszywy i drugi jednak by się wykonał.

mutowalny argument domyślny

Wartość domyślna jest wyliczana *raz*, przy definiowaniu funkcji, więc po `def f(x, L=[])` wszystkie wywołania bez drugiego argumentu dostają tę samą listę: kolejne zwracają `[1]`, `[1, 2]`, `[1, 2, 3]`. To ta sama pułapka referencji co w pierwszej sekcji, tylko widziana od strony wywołań.

ROZDZIAŁ 8

Metody numeryczne

metody numeryczne: Ten dział jest wart tylko cztery punkty, czyli jedno zadanie z dwudziestu pięciu, ale materiału, który autorzy testu naprawdę z niego biorą, jest tak mało, że i tak warto go domknąć. W arkuszu z 17 września 2025 całe zadanie z tego działu dotyczyło wyłącznie rzędów zbieżności oraz założeń metody Newtona i metody siecznych, czyli tematu, od którego zaczynam. Reszta to pojęcia sprawdzane jednym zdaniem: prawie nic nie trzeba tu liczyć, trzeba natomiast wiedzieć, która własność należy do zadania, a która do metody.

8.1 Metody iteracyjne rozwiązywania równań nieliniowych

Mamy równanie $f(x) = 0$, którego nie da się rozwiązać wzorem, więc zamiast szukać pierwiastka wprost, budujemy ciąg przybliżeń. Trzy klasyczne metody różnią się tym, czego wymagają i jak szybko zbliżają się do celu — i o te dwie rzeczy chodzi na teście.

rzęd zbieżności

Bez tego pojęcia cała reszta jest pustą formułką. Oznaczmy przez $e_k = x_k - x^*$ błąd k -tego przybliżenia. Metoda ma rząd zbieżności p , jeżeli błędy kolejnych kroków wiąże zależność

$$|e_{k+1}| \approx C |e_k|^p,$$

czyli nowy błąd jest starym błędem podniesionym do potęgi p . Gdy $p = 1$ (zbieżność liniowa), każdy krok mnoży błąd przez stałą, więc dokładność przyrasta o stałą liczbę cyfr na iterację — równym, ale dowolnym tempem. Gdy $p = 2$ (zbieżność kwadratowa), podnoszenie błędu do kwadratu oznacza, że liczba poprawnych cyfr się *podwaja*: z trzech robi się sześć, potem dwanaście. Różnica między $p = 1$ a $p = 2$ nie jest więc ilościowa, tylko jakościowa.

bisekcja

Zaczynamy od przedziału, na którego końcach funkcja ma różne znaki, więc gdzieś w środku musi przecinać oś; dzielimy przedział na pół i bierzemy tę połówkę, w której zmiana znaku się utrzymała. Wynika stąd wszystko, co o bisekcji trzeba wiedzieć. Jest to *jedyna* z trzech metod z gwarancją zbieżności, bo pierwiastek nigdy nie opuszcza przedziału, a przedział kurczy się do zera — i nie zależy to od tego, jak dobrze zgadliśmy start. Ceną za tę pewność jest wolne tempo: długość przedziału maleje dokładnie dwukrotnie na krok, czyli zbieżność jest liniowa z ilorazem równym $\frac{1}{2}$ — każda iteracja daje jeden bit dokładności. Pochodna nie jest przy tym potrzebna.

metoda Newtona

Pomysł jest geometryczny: w bieżącym punkcie prowadzimy styczną do wykresu i za nowe przybliżenie bierzemy miejsce, w którym ta styczna przecina oś,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Metoda zastępuje więc funkcję jej liniowym przybliżeniem. Skoro takie przybliżenie myli się o człon rzędu kwadratowego, to i błąd zachowuje się kwadratowo — stąd $p = 2$ i podwajanie liczby poprawnych cyfr. Kosztem jest konieczność znania *jawnej pochodnej* w każdym kroku. Najważniejsze jest jednak co innego: ta piękna zbieżność jest wyłącznie *lokalna*. Twierdzenie mówi, że istnieje pewne otoczenie pierwiastka, w którym wszystko działa, i nic nie mówi o tym, co się stanie poza nim.

! Zdanie „metoda Newtona zawsze jest zbieżna, niezależnie od punktu początkowego” jest fałszywe i to jest najczęstsza pułapka działu. Iteracja może wpaść w cykl i skakać w kółko między dwoma punktami, może uciec daleko poza przedział z pierwiastkiem, może wreszcie trafić w punkt o pochodnej bliskiej zeru — wtedy styczna jest niemal pozioma, przecina oś bardzo daleko i nowe przybliżenie jest gorsze od poprzedniego.

Drugie założenie łatwo przeoczyć: pierwiastek musi być *jednokrotny*, czyli $f'(x^*) \neq 0$. Dla pierwiastka wielokrotnego zeruje się nie tylko funkcja, ale i pochodna w mianowniku, a rząd spada z kwadratowego do liniowego.

metoda siecznych

Co zrobić, gdy pochodnej nie znamy albo jest za droga? Zastąpić ją ilorazem różnicowym z dwóch ostatnich punktów, czyli zamiast stycznej poprowadzić sieczną:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Zyskujemy to, że pochodna nie jest potrzebna. Płacimy tym, że sieczna przybliża funkcję gorzej niż styczna, więc zbieżność jest wolniejsza: jej rząd to złoty podział, $p = \frac{1+\sqrt{5}}{2} \approx 1,618$, a więc *mniej* niż 2.

! Stąd druga pułapka: zdanie „metoda siecznych jest zbieżna szybciej niż metoda Newtona przy dobrym punkcie startowym” jest fałszywe — Newton ma rząd 2, sieczne około 1,618.

Dla uczciwości niuans: licząc tempo nie na iterację, lecz na jedno obliczenie funkcji, sieczne mogą wypaść korzystnie, bo krok Newtona wymaga i wartości funkcji, i wartości pochodnej, a krok siecznych tylko jednej nowej wartości. Samo tempo zbieżności pozostaje jednak niższe — i o nie pyta test.

Metoda	Czego wymaga	Rząd	Gwarancja
Bisekcja	zmiany znaku na końcach przedziału	1, iloraz $\frac{1}{2}$	tak, zawsze
Newton	jawnej pochodnej, dobrego startu	2	nie, tylko lokalnie
Sieczne	dwóch punktów startowych	$\approx 1,618$	nie, tylko lokalnie

8.2 Uwarunkowanie zadania i stabilność algorytmu

To rozróżnienie jest czysto pojęciowe i właśnie dlatego autorzy testów je lubią. *Uwarunkowanie* jest własnością samego zadania i mówi, jak bardzo wynik reaguje na małe zaburzenie danych; zadanie źle uwarunkowane to takie, w którym zmiana danych na szesnastym miejscu po przecinku przedstawia wynik na pierwszym. Nie ma to nic wspólnego z metodą ani z komputerem — zadanie jest źle uwarunkowane także wtedy, gdy liczymy je ręcznie i bezbłędnie. *Stabilność* jest natomiast własnością algorytmu i mówi, czy algorytm nie dokłada własnych błędów ponad to, co wymusza samo zadanie.

wniosek, o który chodzi na teście: Nawet doskonale stabilny algorytm zastosowany do zadania źle uwarunkowanego da wynik obciążony dużym błędem, bo ten błąd siedzi w *zadaniu*, a nie w metodzie. Zmiana metody nic tu nie pomoże; pomogą tylko dokładniejsze dane.

wskaźnik uwarunkowania

Miarą uwarunkowania układu równań liniowych jest iloczyn normy macierzy i normy jej odwrotności, $\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$, który z grubsza mówi, ile razy błąd względny danych może się powiększyć w wyniku. Klasycznym przykładem macierzy fatalnie uwarunkowanej jest *macierz Hilberta* o wyrazach $1/(i+j-1)$. Osobnym pojęciem jest *algorytm numerycznie poprawny* — taki, którego wynik jest *dokładnym* rozwiązaniem zadania z lekko zaburzonymi danymi. Definicja jest sprytna: nie obiecuje wyniku bliskiego prawdziwemu, bo przy złym uwarunkowaniu obiecać się tego nie da, a tylko tyle, że algorytm nie zrobił nic gorszego niż drobne pogorszenie danych.

8.3 Utrata cyfr znaczących

Odejmowanie dwóch bliskich sobie liczb jest operacją niebezpieczną. Wspólne cyfry początkowe kasują się i znikają z wyniku, a to, co zostaje, pochodzi z cyfr końcowych — czyli z tych, które były już zaokrąglone. Błąd bezwzględny się nie zmienia, ale wynik jest teraz mały, więc błąd *względny* rośnie gwałtownie: odejmując liczby zgodne na ośmiu cyfrach, z szesnastu cyfr wiarygodnych robimy osiem.

Podręcznikowym przykładem jest wzór na pierwiastki równania kwadratowego, gdy b^2 jest znacznie większe od $4ac$. Wtedy $\sqrt{b^2 - 4ac} \approx |b|$ i w jednym z dwóch wzorów odejmujemy od siebie prawie identyczne liczby — ten pierwiastek wychodzi niedokładny, choć drugi jest policzony dobrze. Ratunek: policzyć „bezpieczny” pierwiastek, ten liczony przez dodawanie, a drugi wyznaczyć nie z odejmowania, tylko ze wzoru na iloczyn pierwiastków, $x_1 x_2 = c/a$.

Miarą dokładności arytmetyki jest epsilon maszynowy, w podwójnej precyzji rzędu 10^{-16} ; uczciwie trzeba dodać, że spotyka się dwie konwencje — 2^{-52} jako odstęp między jedyneką a następną liczbą maszynową oraz 2^{-53} jako maksymalny błąd względny zaokrąglenia. Skończona precyzja sprawia też, że dodawanie liczb zmiennoprzecinkowych *nie jest łączne*: $(1 + \varepsilon/2) + \varepsilon/2$ daje 1, bo każde dodanie z osobna zaokrągla się w dół, natomiast $1 + (\varepsilon/2 + \varepsilon/2)$ daje $1 + \varepsilon$.

8.4 Całkowanie numeryczne

Pomysł kwadratury jest zawsze ten sam: nie umiemy scałkować funkcji dokładnie, więc dzielimy przedział na małe kawałki, na każdym zastępujemy funkcję czymś prostym, co scałkować umiemy, i sumujemy wyniki. Jeżeli tym prostym jest odcinek, dostajemy wzór trapezów; jeżeli parabola przez trzy punkty — wzór

Simpsona. Dla wzorów złożonych o kroku siatki h błąd trapezów jest rzędu h^2 , a Simpsona rzędu h^4 , czyli dwukrotne zagęszczenie siatki zmniejsza błąd trapezów *czterokrotnie*, a błąd Simpsona *szesnastokrotnie*.

Drugą, niezależną miarą jakości wzoru jest stopień dokładności, czyli najwyższy stopień wielomianu, dla którego wzór daje wynik *dokładny*. Trapezy mają stopień 1. Wzór Simpsona powstaje z przybliżenia paraboli, więc oczekiwalibyśmy stopnia 2 — tymczasem jest dokładny także dla wielomianów stopnia 3, bo dzięki symetrii węzłów człon sześcienny błędu znosi się sam.

! To miła niespodzianka i typowa pułapka: zdanie „wzór Simpsona jest dokładny dla wielomianów stopnia co najwyżej 2” jest fałszywe. Prawidłowa odpowiedź to stopień 3.

Trapezy i Simpson mają węzły ustawione z góry i dobierają tylko wagi. Gauss zauważył, że optymalizować można także *położenia węzłów*, co przy n węzłach daje $2n$ swobodnych parametrów zamiast n . Kwadratura Gaussa-Legendre’a o n węzłach jest więc dokładna dla wielomianów stopnia $2n-1$ — już dwa węzły całkują dokładnie wielomian stopnia trzeciego. Osobnym pomysłem jest metoda Romberga: liczymy trapezami na dwóch siatkach, rzadszej i dwa razy gęstszej, a potem łączymy wyniki tak, żeby główny człon błędu się skasował.

8.5 Interpolacja wielomianowa

Podstawowy fakt brzmi: przez $n+1$ punktów o *różnych* węzłach przechodzi dokładnie jeden wielomian stopnia *co najwyżej* n . Skoro jest jeden, to postać Lagrange’a i postać Newtona nie mogą być różnymi wielomianami — to dwa zapisy tej samej rzeczy, różniące się tylko wygodą.

! Słowa „co najwyżej” nie wolno zgubić — tam właśnie stawia się pułapkę. Zdanie „przez $n+1$ punktów przechodzi dokładnie jeden wielomian stopnia n ” jest fałszywe, bo punkty mogą leżeć na wielomianie niższego stopnia: trzy punkty współliniowe wyznaczają prostą, a nie parabolę.

Oczekiwalibyśmy, że im więcej węzłów, tym lepsze przybliżenie. Dla węzłów *równoodległych* bywa odwrotnie: wielomian dobrze trzyma się funkcji w środku przedziału, ale przy końcach zaczyna falować, a amplituda oscylacji rośnie z liczbą węzłów, zamiast maleć. To jest efekt Rungego, a lekarstwem są węzły Czebyszewa — zagęszczone przy brzegach, rzadsze w środku — które ten efekt usuwają.

8.6 Rozkład LU i po co się go robi

Rozkład $A = LU$ na macierz dolnotrójkątną i górnątrojkątną to zapisana eliminacja Gaussa i kosztuje rzędu n^3 operacji — tyle samo co zwykle rozwiązanie układu. Zysk przychodzi dopiero przy kolejnym układzie: mając gotowe L i U , rozwiązanie z *nową* prawą stroną to już tylko dwa układy trójkątne, czyli rzędu n^2 operacji. Gdy tę samą macierz trzeba rozwiązać z wieloma prawymi stronami, opłaca się więc rozłożyć ją raz. Przy okazji wyznacznik dostajemy za darmo — jest iloczynem elementów na przekątnej.

Piwotowanie, czyli przestawianie wierszy tak, by na przekątnej znalazł się element możliwie duży co do modułu, ma dwa powody i drugi jest ważniejszy. Gdy na przekątnej wypadnie zero, dzielenie jest niewykonalne. Gdy wypadnie liczba bardzo mała, dzielenie wykona się bez przeszkód, ale przemnoży dotychczasowe błędy zaokrągleń przez ogromny czynnik. Piwotowanie służy więc głównie *stabilności*, a nie wykonalności. Dla macierzy symetrycznych i dodatnio określonych stosuje się zresztą tańszy o połowę rozkład Cholesky’ego, $A = LL^T$, który wykorzystuje symetrię.